Data Analytics Assignment 2

Group 22
Names : Aadharsh Ramachandran(231IT001), Jaivant Kosaraju(231IT028), K Akhilesh(231IT029), Paluvadi Dinesh Manideep(231IT047)

Question :

For the dataset you chose earlier (or the dataset used in your base paper if suitable),

1. Design a data preprocessing pipeline to statistically identify the type of missing data. Once identified, apply various missing value techniques including multiple imputation techniques and observe the changes in analysis. Apply any applicable discretization techniques to handle outliers, and appropriate scaling/normalization techniques, to handle noise and inconsistency in your dataset, clearly justifying each decision.
2. Implement different correlation analysis techniques to study the relationships between variables to identify potential direct, partial or multiple correlation. Also, check if covariance exists.
3. Apply suitable dimensionality reduction techniques such as Principal Component Analysis (PCA), report the explained variance, and interpret the retained components.
4. Perform feature engineering by creating new meaningful features and explain their relevance.

Upload a detailed report presenting your observations and insights, highlighting how preprocessing and advanced modeling techniques influenced the results (with your implemented code snippets and supporting visualisations if applicable)

Contributions :
SubGroup 1 : Jaivant Kosaraju and Paluvadi Dinesh Manideep
Paluvadi DInesh Manideep: Implemented list-wise, regression and KNN, applied Quantile and Power transformers, ran PCA, performed equal-width discretization, ran Pearson analyses and multiple correlation and created BMI, Age_months, and Age_Group.
Jaivant Kosaraju: Implemented MICE and cold-deck variants, applied MaxAbs and Standard scaler, performed equal-depth discretization, ran Spearman correlation analyses and multiple correlation, ran PCA, and created Age_BMI_Interaction, nutrition_score, and health_risk.

SubGroup 2 : Aadharsh Ramachandran and K Akhilesh
K Akhilesh: Implemented sequential/LOCF and distribution-based and EM imputations, performed supervised discretization, applied Robust scalers, ran scatter correlation checks and partial correlation , and created BP_systolic_avg, BP_diastolic_avg, and Pulse_avg.
Aadharsh Ramachandran: Implemented missing value analysis and hot-deck imputation, applied unit-vector and minmax normalizations, performed K-means clustering, ran covariance, partial correlation, regression analyses and ICA and created BP_Difference and Pulse_Rate_Ratio.

Code Links :
SubGroup 1 :
https://www.kaggle.com/code/dineshmanideep/dataanalyticsassignment-2-group-1
SubGroup 2 :
https://www.kaggle.com/code/aadharshramachandran/data-analytics-assignment-2-group-2

Dataset details :

In this work, we used three district-level datasets from the CAB 2014 NationalHealth and Nutrition Survey, specifically for Agra, Indore, and Mathura districts. Each dataset contains individual-level clinical, anthropometric, and biochemical measurements collected across both children and adults. The variables include height, weight, body mass index (BMI),haemoglobin levels, blood pressure, blood glucose (where available), and other health and nutrition indicators. These measurements allow me to study nutritional status and health conditions such as anemia, undernutrition, overweight/obesity, and early signs of non-communicable disease risk at the district level.

Sources:

1) https://aikosh.indiaai.gov.in/home/datasets/details/cab_2014_survey_agra_district_uttar_pradesh.html
2) https://aikosh.indiaai.gov.in/home/datasets/details/mathura_district_uttar_pradesh_cab_2014_survey.html
3) https://aikosh.indiaai.gov.in/home/datasets/details/indore_madhya_pradesh_cab_2014_survey.html

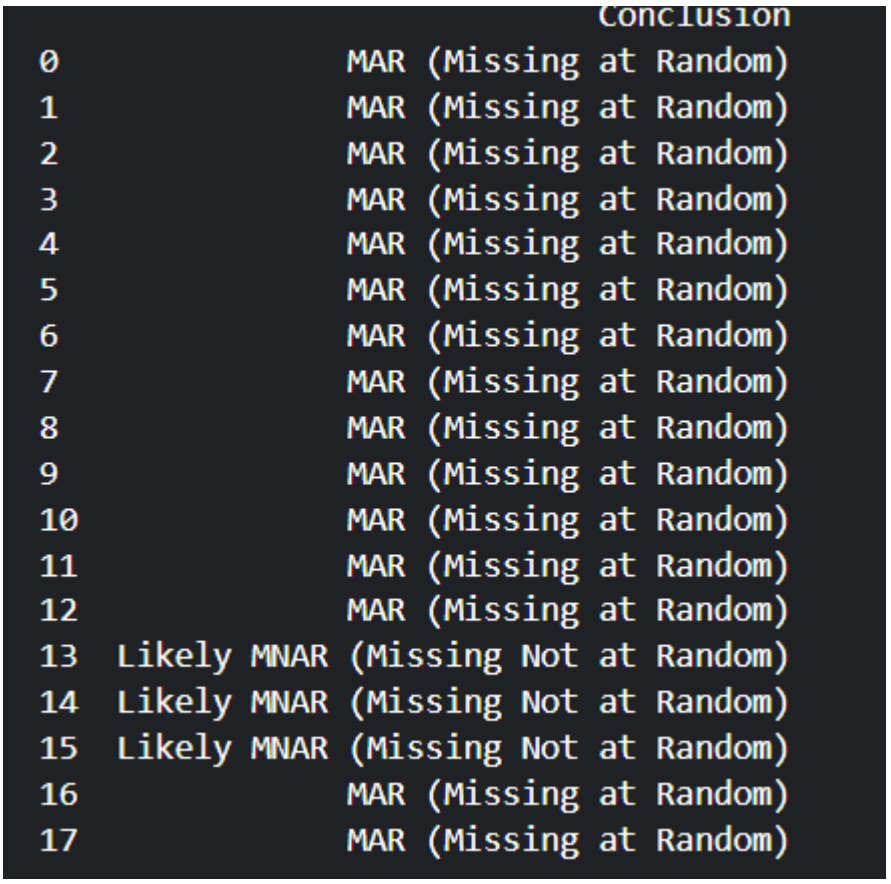
Identifying missing value types:

```
df_clean=df.dropna(axis=1,how="all")
print("--- GLOBAL TEST: Little's MCAR Test ---")
try:
    mcar_test=MCARTest(method="little")
    p_little=mcar_test.little_mcar_test(df_clean)
    print(f"Little's MCAR Test p-value: {p_little:.4f}")
    global_mcar=p_little>0.05
    if global_mcar:
        print("Global Result: Missingness is likely MCAR (Completely Random).")
    else:
        print("Global Result: Missingness is NOT MCAR (Suggests MAR or MNAR).")
except Exception as e:
    print(f"Little's Test failed: {e}")
    global_mcar=False
print("COLUMN-BY-COLUMN ANALYSIS")
summary_results=[]
numeric_cols=df_clean.select_dtypes(include=[np.number]).columns.tolist()
categorical_cols=df_clean.select_dtypes(exclude=[np.number]).columns.tolist()
for target_col in df_clean.columns:
    missing_count=df_clean[target_col].isnull().sum()
```

```
    if missing_count==0:
        continue
    y=df_clean[target_col].isnull().astype(int)
    X=df_clean.drop(columns=[target_col]).copy()
    for col in X.columns:
        if X[col].dtype in [np.float64, np.int64]:
            X[col]=X[col].fillna(X[col].median())
        else:
            X[col]=X[col].fillna(X[col].mode()[0] if not X[col].mode().empty else "Unknown")
    X_numeric=pd.get_dummies(X, drop_first=True)
    X_numeric=sm.add_constant(X_numeric)
    try:
        model=sm.Logit(y, X_numeric).fit(disp=False)
        significant_predictors=(model.pvalues[1:]<0.05).any()
    except:
        significant_predictors=False
    if significant_predictors:
        conclusion="MAR (Missing at Random)"
    elif global_mcar:
        conclusion="MCAR (Missing Completely at Random)"
    else:
        conclusion="Likely MNAR (Missing Not at Random)"
    summary_results.append({"Column":target_col,"Missing
Count":missing_count,"Conclusion":conclusion})
summary_df=pd.DataFrame(summary_results)
print("FINAL MISSINGNESS SUMMARY")
print(summary_df)
```

```
--- GLOBAL TEST: Little's MCAR Test ---
Little's MCAR Test p-value: 0.0000
Global Result: Missingness is NOT MCAR (Suggests MAR or MNAR).
COLUMN-BY-COLUMN ANALYSIS
FINAL MISSINGNESS SUMMARY
```

	Column	Missing Count
0	Weight_in_kg	3828
1	Length_height_cm	3856
2	Haemoglobin_level	10407
3	BP_systolic	10084
4	BP_systolic_2_reading	10084
5	BP_Diastolic	10084
6	BP_Diastolic_2reading	10084
7	Pulse_rate	10084
8	Pulse_rate_2_reading	10084
9	fasting_blood_glucose_mg_dl	11623
10	duration_pregnanacy	21595
11	day_or_month_for_breast_feeding	21238
12	water_month	21238
13	ani_milk_month	21238
14	semisolid_month_or_day	21238
15	solid_month	21238
16	vegetables_month_or_day	21238
17	illness_duration	21627



	Conclusion
0	MAR (Missing at Random)
1	MAR (Missing at Random)
2	MAR (Missing at Random)
3	MAR (Missing at Random)
4	MAR (Missing at Random)
5	MAR (Missing at Random)
6	MAR (Missing at Random)
7	MAR (Missing at Random)
8	MAR (Missing at Random)
9	MAR (Missing at Random)
10	MAR (Missing at Random)
11	MAR (Missing at Random)
12	MAR (Missing at Random)
13	Likely MNAR (Missing Not at Random)
14	Likely MNAR (Missing Not at Random)
15	Likely MNAR (Missing Not at Random)
16	MAR (Missing at Random)
17	MAR (Missing at Random)

1. MAR (Missing At Random):

Most variables such as health measurements and survey data are MAR because the missing values depend on other recorded variables or whether the measurement/test was conducted .

Columns: Weight_in_kg , Length_height_cm , Haemoglobin_level , BP_systolic , BP_systolic_2_reading , BP_Diastolic , BP_Diastolic_2reading , Pulse_rate , Pulse_rate_2_reading , fasting_blood_glucose_mg_dl , duration_pregnanacy , day_or_month_for_breast_feeding , water_month , vegetables_month_or_day , illness_duration .

2. MNAR (Missing Not At Random):

Some feeding-related variables are MNAR because the missing values likely depend on the actual value itself or recall/reporting behavior of respondents .

Columns: ani_milk_month , semisolid_month_or_day , solid_month .

Missing value techniques:

a) **Listwise deletion**

```
print("List-wise Deletion")
df_listwise_deleted = df.dropna()
rows_original=len(df)
rows_after_deletion=len(df_listwise_deleted)
rows_removed=rows_original-rows_after_deletion
print(f"Original DataFrame rows: {rows_original}")
print(f"DataFrame rows after list-wise deletion: {rows_after_deletion}")
print(f"Number of rows removed: {rows_removed}")
```

```
print(f"Percentage of rows removed: {((rows_removed/rows_original)*100):.2f}%")
selected_columns=['Age','Weight_in_kg','Length_height_cm','Haemoglobin_level']
print("Descriptive Statistics for Original df (selected columns):")
print(df[selected_columns].describe())
print("Descriptive Statistics for df_listwise_deleted (selected columns):")
print(df_listwise_deleted[selected_columns].describe())
```

```
List-wise Deletion
Original DataFrame rows: 22099
DataFrame rows after list-wise deletion: 0
Number of rows removed: 22099
Percentage of rows removed: 100.00%
Descriptive Statistics for Original df (selected columns):
                Age  Weight_in_kg  Length_height_cm  Haemoglobin_level
count  22099.000000  18271.000000      18243.000000       11692.000000
mean      28.832662     40.591198        137.012333           9.911093
std       19.435118     23.237620         33.539693           2.179684
min        1.000000      1.000000         45.200001           3.000000
25%       14.000000     22.500000        124.200000           8.500000
50%       24.000000     45.299999        150.000000          10.000000
75%       42.000000     54.799999        159.600010          11.200000
max       99.000000    960.299990        198.000000          17.000000
Descriptive Statistics for df_listwise_deleted (selected columns):
       Age  Weight_in_kg  Length_height_cm  Haemoglobin_level
count  0.0           0.0               0.0                0.0
mean   NaN           NaN               NaN                NaN
std    NaN           NaN               NaN                NaN
min    NaN           NaN               NaN                NaN
25%    NaN           NaN               NaN                NaN
50%    NaN           NaN               NaN                NaN
75%    NaN           NaN               NaN                NaN
max    NaN           NaN               NaN                NaN
```

b) Deleting Columns

A 90.0% threshold was chosen because columns with such a high proportion of missing values provide very little information and could introduce significant bias or noise if imputed. Deleting them is often a pragmatic approach when the missingness is so extensive, as imputation would likely involve fabricating a majority of the data points, which could distort the true underlying distributions and relationships within the dataset.

```
print("Missing Data Handling - Deleting Columns")
threshold=90.0
missing_percentages=(df.isnull().sum()/len(df))*100
missing_percentages=missing_percentages.sort_values(ascending=False)
columns_to_drop=missing_percentages[missing_percentages>threshold].index.tolist(
)
df_cols_deleted=df.drop(columns=columns_to_drop)
print(f"Chosen missing value threshold: {threshold}%")
print(f"Columns identified for deletion (missing>{threshold}%):{columns_to_drop}")
print(f"Original DataFrame shape: {df.shape}")
```

```
print(f"New DataFrame (df_cols_deleted) shape after dropping columns:
{df_cols_deleted.shape}")
```

```
Missing Data Handling - Deleting Columns
Chosen missing value threshold: 90.0%
Columns identified for deletion (missing>90.0%):['illness_duration', 'duration_pregnanacy', 'semisolid_month_or_da
y', 'day_or_month_for_breast_feeding', 'vegetables_month_or_day', 'solid_month', 'ani_milk_month', 'water_month']
Original DataFrame shape: (22099, 24)
New DataFrame (df_cols_deleted) shape after dropping columns: (22099, 16)
```

c) Pairwise deletion

Pairwise deletion is a method for handling missing data that excludes a case only if it lacks data for the specific pair of variables being analyzed, rather than removing the entire row.Here since we are not analysing or calculating something separately and analysing it and we are analysing the dataset as a whole, no pairwise deletion implemented.

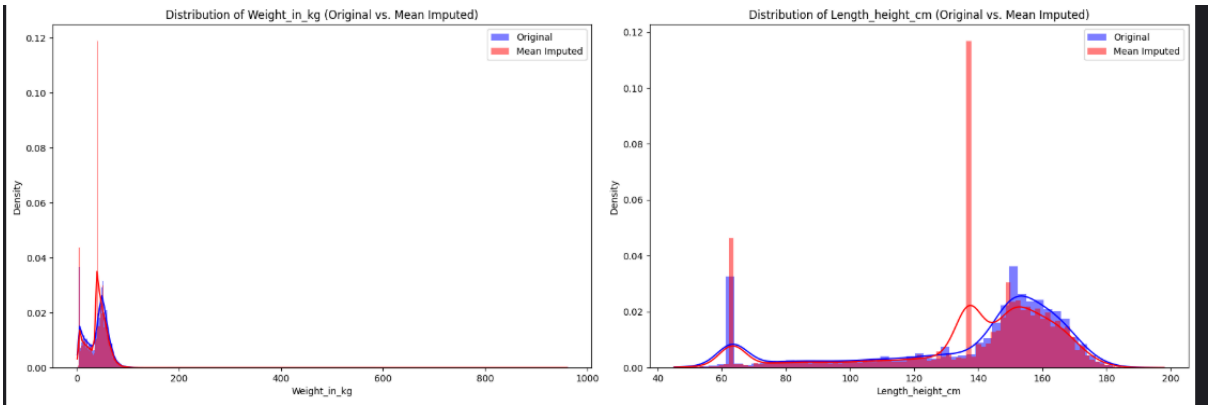
d) Cold-deck imputation

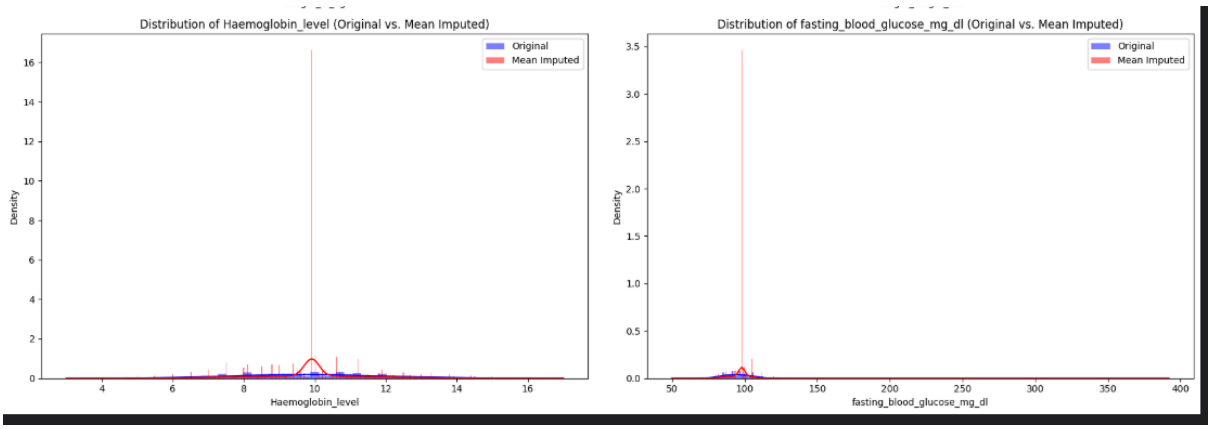
```
print("Missing Data Handling-Cold-deck imputation (Mean, Median, Mode)")
numerical_cols_with_missing=[col for col in df_cols_deleted.columns if
df_cols_deleted[col].isnull().any() and
pd.api.types.is_numeric_dtype(df_cols_deleted[col])]
print(f"Numerical columns with missing values identified for imputation:
{numerical_cols_with_missing}")
df_mean_imputed=df_cols_deleted.copy()
for col in numerical_cols_with_missing:
    mean_value=df_mean_imputed[col].mean()
    df_mean_imputed[col].fillna(mean_value,inplace=True)
df_median_imputed=df_cols_deleted.copy()
for col in numerical_cols_with_missing:
    median_value=df_median_imputed[col].median()
    df_median_imputed[col].fillna(median_value,inplace=True)
df_mode_imputed=df_cols_deleted.copy()
for col in numerical_cols_with_missing:
    mode_value=df_mode_imputed[col].mode()[0]
    df_mode_imputed[col].fillna(mode_value,inplace=True)
print("Shape of df_cols_deleted (original after column deletion):",
df_cols_deleted.shape)
print("Shape of df_mean_imputed:", df_mean_imputed.shape)
print("Shape of df_median_imputed:", df_median_imputed.shape)
print("Shape of df_mode_imputed:", df_mode_imputed.shape)
print("Missing values in df_mean_imputed (selected columns):\n",
df_mean_imputed[numerical_cols_with_missing].isnull().sum())
print("Missing values in df_median_imputed (selected columns):\n",
df_median_imputed[numerical_cols_with_missing].isnull().sum())
print("Missing values in df_mode_imputed (selected columns):\n",
df_mode_imputed[numerical_cols_with_missing].isnull().sum())
selected_numerical_cols=['Weight_in_kg','Length_height_cm','Haemoglobin_level','fa
sting_blood_glucose_mg_dl']
plt.figure(figsize=(18,12))
for i, col in enumerate(selected_numerical_cols):
```

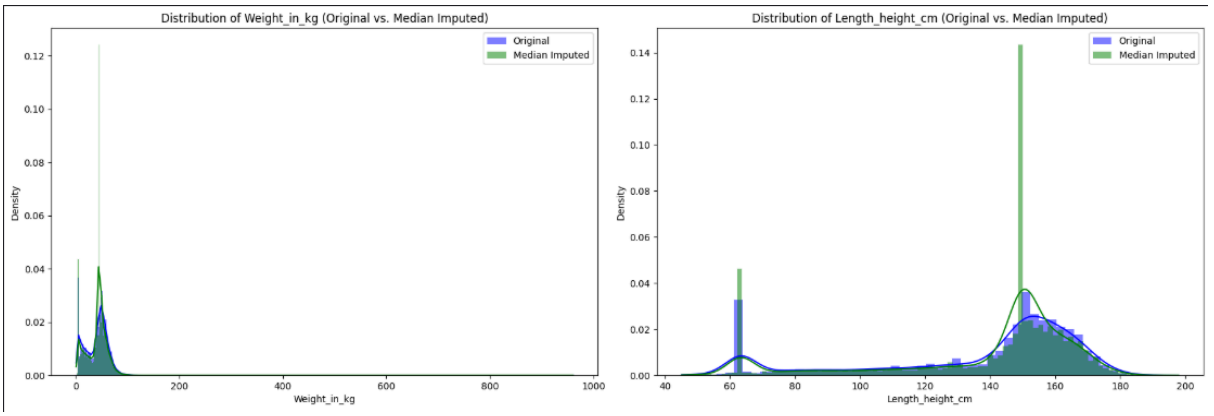
```
    plt.subplot(2,2,i+1)
sns.histplot(df[col].dropna(),color='blue',label='Original',kde=True,stat='density',linewi
dth=0,alpha=0.5)
    sns.histplot(df_mean_imputed[col],color='red',label='Mean
Imputed',kde=True,stat='density',linewidth=0,alpha=0.5)
    plt.title(f'Distribution of {col} (Original vs. Mean Imputed)')
    plt.legend()
plt.tight_layout()
plt.show()
print("Distribution plots for original and mean-imputed data displayed.")
plt.figure(figsize=(18,12))
for i, col in enumerate(selected_numerical_cols):
    plt.subplot(2,2,i+1)
sns.histplot(df[col].dropna(),color='blue',label='Original',kde=True,stat='density',linewi
dth=0,alpha=0.5)
    sns.histplot(df_median_imputed[col],color='green',label='Median
Imputed',kde=True,stat='density',linewidth=0,alpha=0.5)
    plt.title(f'Distribution of {col} (Original vs. Median Imputed)')
    plt.legend()
plt.tight_layout()
plt.show()
print("Distribution plots for original and median-imputed data displayed.")
plt.figure(figsize=(18,12))
for i, col in enumerate(selected_numerical_cols):
    plt.subplot(2,2,i + 1)
sns.histplot(df[col].dropna(),color='blue',label='Original',kde=True,stat='density',linewi
dth=0,alpha=0.5)
    sns.histplot(df_mode_imputed[col],color='orange',label='Mode
Imputed',kde=True,stat='density',linewidth=0,alpha=0.5)
    plt.title(f'Distribution of {col} (Original vs. Mode Imputed)')
    plt.legend()
plt.tight_layout()
plt.show()
print("Distribution plots for original and mode-imputed numerical data displayed.")
```

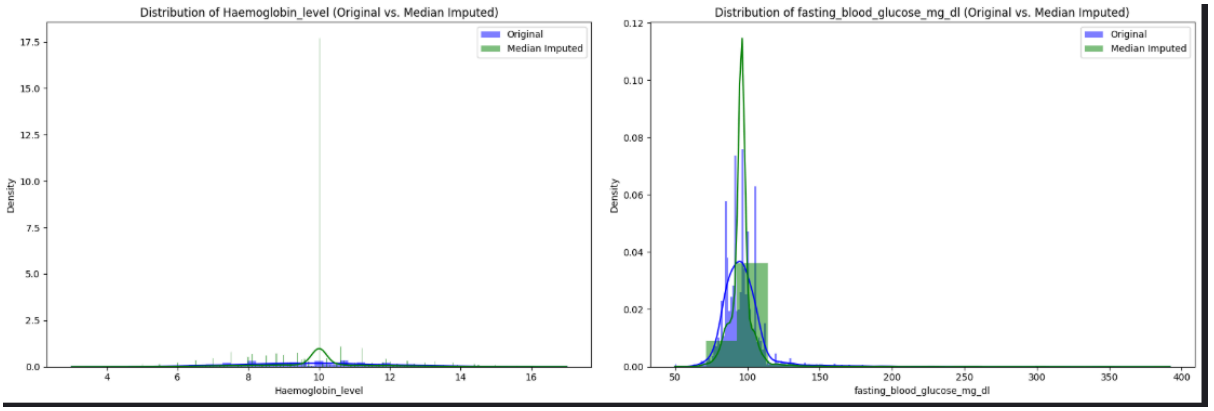
```
Missing Data Handling-Cold-deck imputation (Mean, Median, Mode)
Numerical columns with missing values identified for imputation: ['Weight_in_kg', 'Length_height_cm', 'Haemoglobin
_level', 'BP_systolic', 'BP_systolic_2_reading', 'BP_Diastolic', 'BP_Diastolic_2reading', 'Pulse_rate', 'Pulse_rat
e_2_reading', 'fasting_blood_glucose_mg_dl']
Shape of df_cols_deleted (original after column deletion): (22099, 16)
Shape of df_mean_imputed: (22099, 16)
Shape of df_median_imputed: (22099, 16)
Shape of df_mode_imputed: (22099, 16)
Missing values in df_mean_imputed (selected columns):
 Weight_in_kg                0
Length_height_cm             0
Haemoglobin_level            0
BP_systolic                  0
BP_systolic_2_reading        0
BP_Diastolic                 0
BP_Diastolic_2reading        0
Pulse_rate                   0
Pulse_rate_2_reading         0
fasting_blood_glucose_mg_dl  0
dtype: int64
Missing values in df_median_imputed (selected columns):
 Weight_in_kg                0
Length_height_cm             0
Haemoglobin_level            0
BP_systolic                  0
BP_systolic_2_reading        0
BP_Diastolic                 0
```

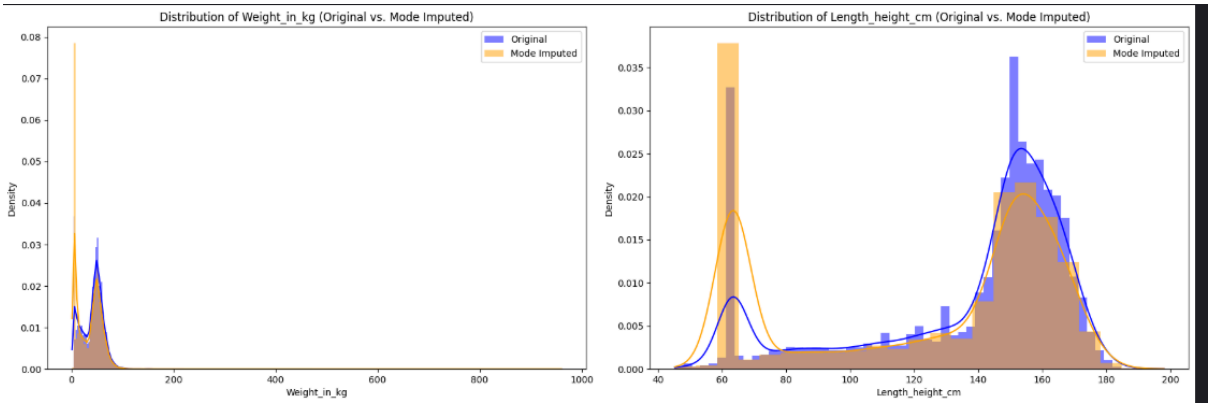
```
BP_Diastolic_2reading        0
Pulse_rate                   0
Pulse_rate_2_reading         0
fasting_blood_glucose_mg_dl  0
dtype: int64
Missing values in df_mode_imputed (selected columns):
 Weight_in_kg                0
Length_height_cm             0
Haemoglobin_level            0
BP_systolic                  0
BP_systolic_2_reading        0
BP_Diastolic                 0
BP_Diastolic_2reading        0
Pulse_rate                   0
Pulse_rate_2_reading         0
fasting_blood_glucose_mg_dl  0
dtype: int64
```

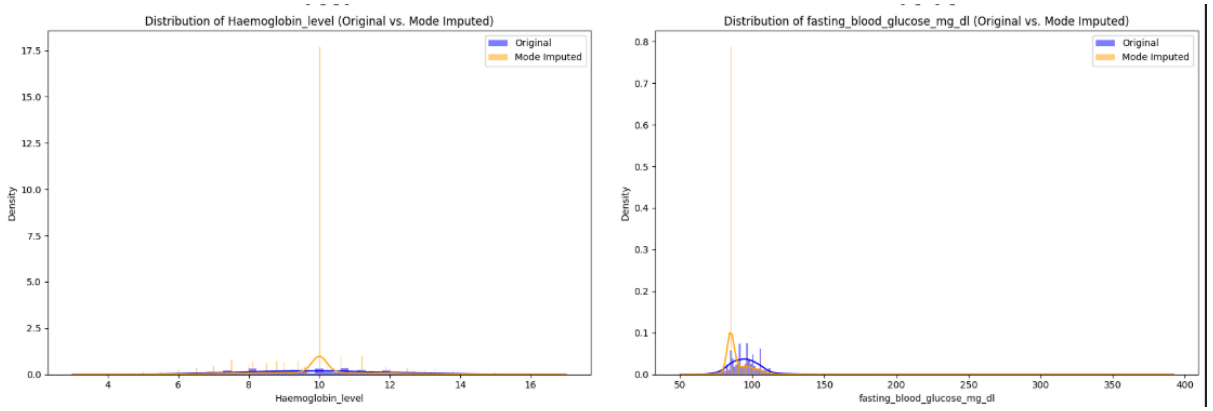



Distribution of Haemoglobin_level (Original vs. Mean Imputed)
Original
Mean Imputed
Density
Haemoglobin_level
Distribution of fasting_blood_glucose_mg_dl (Original vs. Mean Imputed)
Original
Mean Imputed
Density
fasting_blood_glucose_mg_dl


Distribution of Weight_in_kg (Original vs. Median Imputed)
Original
Median Imputed
Density
Weight_in_kg
Distribution of Length_height_cm (Original vs. Median Imputed)
Original
Median Imputed
Density
Length_height_cm


Distribution of Haemoglobin_level (Original vs. Median Imputed)
Original
Median Imputed
Density
Haemoglobin_level
Distribution of fasting_blood_glucose_mg_dl (Original vs. Median Imputed)
Original
Median Imputed
Density
fasting_blood_glucose_mg_dl


Distribution of Weight_in_kg (Original vs. Mode Imputed)
Original
Mode Imputed
Density
Weight_in_kg
Distribution of Length_height_cm (Original vs. Mode Imputed)
Original
Mode Imputed
Density
Length_height_cm



e) Hot-deck imputation

```
print("## Missing Data Handling - Hot-Deck Imputation (Conceptual Code)")
df_hotdeck_imputed=df_cols_deleted.copy()
target_col='Weight_in_kg'
matching_cols=['Age', 'Length_height_cm']
print(f"Target column for Hot-Deck imputation: '{target_col}'")
print(f"Matching columns for similarity: {matching_cols}")
for col in matching_cols:
    if df_hotdeck_imputed[col].isnull().any():
        median_val=df_hotdeck_imputed[col].median()
        df_hotdeck_imputed[col]=df_hotdeck_imputed[col].fillna(median_val)
        print(f"Missing values in matching column '{col}' imputed with median:
{median_val:.2f}")
missing_weight_rows=df_hotdeck_imputed[df_hotdeck_imputed[target_col].isna()]
donor_rows=df_hotdeck_imputed[df_hotdeck_imputed[target_col].notna()]
print(f"Number of rows with missing '{target_col}': {len(missing_weight_rows)}")
print(f"Number of potential donor rows: {len(donor_rows)}")
missing_indices=missing_weight_rows.index
print("DataFrame prepared for conceptual Hot-Deck imputation.")
scaler=StandardScaler()
scaled_donor_matching_cols=scaler.fit_transform(donor_rows[matching_cols])
scaled_missing_matching_cols=scaler.transform(missing_weight_rows[matching_col
s])
scaled_donor_df=pd.DataFrame(scaled_donor_matching_cols,
columns=matching_cols,index=donor_rows.index)
scaled_missing_df=pd.DataFrame(scaled_missing_matching_cols,
columns=matching_cols,index=missing_weight_rows.index)
sample_missing_indices=missing_indices[:50]
print(f"Performing Hot-Deck imputation for {len(sample_missing_indices)} sample
rows...")
imputed_examples=[]
for idx in sample_missing_indices:
    current_missing_features=scaled_missing_df.loc[idx,matching_cols].values
    distances=scaled_donor_df.apply(lambda row:
euclidean(current_missing_features,row[matching_cols].values),axis=1)
    most_similar_donor_idx=distances.idxmin()
    imputed_value=donor_rows.loc[most_similar_donor_idx,target_col]
```

```
    df_hotdeck_imputed.loc[idx, target_col]=imputed_value
    imputed_examples.append({'Missing Index':idx,
      'Original Age':df_hotdeck_imputed.loc[idx,'Age'],
      'Original Length_height_cm':df_hotdeck_imputed.loc[idx,'Length_height_cm'],
      'Imputed Weight_in_kg':imputed_value,
      'Donor Index':most_similar_donor_idx,
      'Donor Weight_in_kg':donor_rows.loc[most_similar_donor_idx, target_col],
      'Donor Age':donor_rows.loc[most_similar_donor_idx,'Age'],
      'Donor
Length_height_cm':donor_rows.loc[most_similar_donor_idx,'Length_height_cm']
    })
print("Examples of Hot-Deck Imputation:")
for example in imputed_examples[:5]:
    print(f"Missing Row (idx={example['Missing Index']}): Age={example['Original
Age']:.1f}, Length={example['Original Length_height_cm']:.1f}, Imputed
Weight={example['Imputed Weight_in_kg']:.1f}")
    print(f"  -> Donor Row (idx={example['Donor Index']}): Age={example['Donor
Age']:.1f}, Length={example['Donor Length_height_cm']:.1f}, Weight={example['Donor
Weight_in_kg']:.1f}")
print(f"\nMissing values in '{target_col}' after sample Hot-Deck imputation:
{df_hotdeck_imputed.loc[sample_missing_indices, target_col].isnull().sum()}")
```

```
## Missing Data Handling - Hot-Deck Imputation (Conceptual Code)
Target column for Hot-Deck imputation: 'Weight_in_kg'
Matching columns for similarity: ['Age', 'Length_height_cm']
Missing values in matching column 'Length_height_cm' imputed with median: 150.00
Number of rows with missing 'Weight_in_kg': 3828
Number of potential donor rows: 18271
DataFrame prepared for conceptual Hot-Deck imputation.
Performing Hot-Deck imputation for 50 sample rows...
Examples of Hot-Deck Imputation:
Missing Row (idx=562): Age=17.0, Length=150.0, Imputed Weight=40.4
  -> Donor Row (idx=14384): Age=17.0, Length=150.0, Weight=40.4
Missing Row (idx=572): Age=10.0, Length=150.0, Imputed Weight=23.3
  -> Donor Row (idx=10586): Age=10.0, Length=150.0, Weight=23.3
Missing Row (idx=765): Age=7.0, Length=150.0, Imputed Weight=51.0
  -> Donor Row (idx=12763): Age=7.0, Length=148.4, Weight=51.0
Missing Row (idx=766): Age=6.0, Length=150.0, Imputed Weight=18.3
  -> Donor Row (idx=10192): Age=6.0, Length=148.6, Weight=18.3
Missing Row (idx=767): Age=3.0, Length=150.0, Imputed Weight=11.3
  -> Donor Row (idx=20500): Age=3.0, Length=150.0, Weight=11.3

Missing values in 'Weight_in_kg' after sample Hot-Deck imputation: 0
```

f) LOCF(Last Observation Carried Forward) and NOCB(Next Observation Carried Backward)

```
print("Missing Data Handling - Sequential Imputation (LOCF, NOCB)")
df_locf_imputed=df_cols_deleted.copy()
df_locf_imputed[numerical_cols_with_missing]=df_locf_imputed[numerical_cols_with
_missing].ffill()
df_nocb_imputed=df_cols_deleted.copy()
```

```
df_nocb_imputed[numerical_cols_with_missing]=df_nocb_imputed[numerical_cols_with_missing].bfill()
print("Missing values in df_locf_imputed (selected numerical columns after LOCF):", df_locf_imputed[numerical_cols_with_missing].isnull().sum())
print("Missing values in df_nocb_imputed (selected numerical columns after NOCB):", df_nocb_imputed[numerical_cols_with_missing].isnull().sum())
```

```
Missing Data Handling - Sequential Imputation (LOCF, NOCB)
Missing values in df_locf_imputed (selected numerical columns after LOCF):
 Weight_in_kg                  0
Length_height_cm               0
Haemoglobin_level            121
BP_systolic                  119
BP_systolic_2_reading        119
BP_Diastolic                 119
BP_Diastolic_2reading        119
Pulse_rate                   119
Pulse_rate_2_reading         119
fasting_blood_glucose_mg_dl  119
dtype: int64
Missing values in df_nocb_imputed (selected numerical columns after NOCB):
 Weight_in_kg                  0
Length_height_cm               0
Haemoglobin_level              0
BP_systolic                    6
BP_systolic_2_reading          6
BP_Diastolic                   6
BP_Diastolic_2reading          6
Pulse_rate                     6
Pulse_rate_2_reading           6
fasting_blood_glucose_mg_dl    6
dtype: int64
```

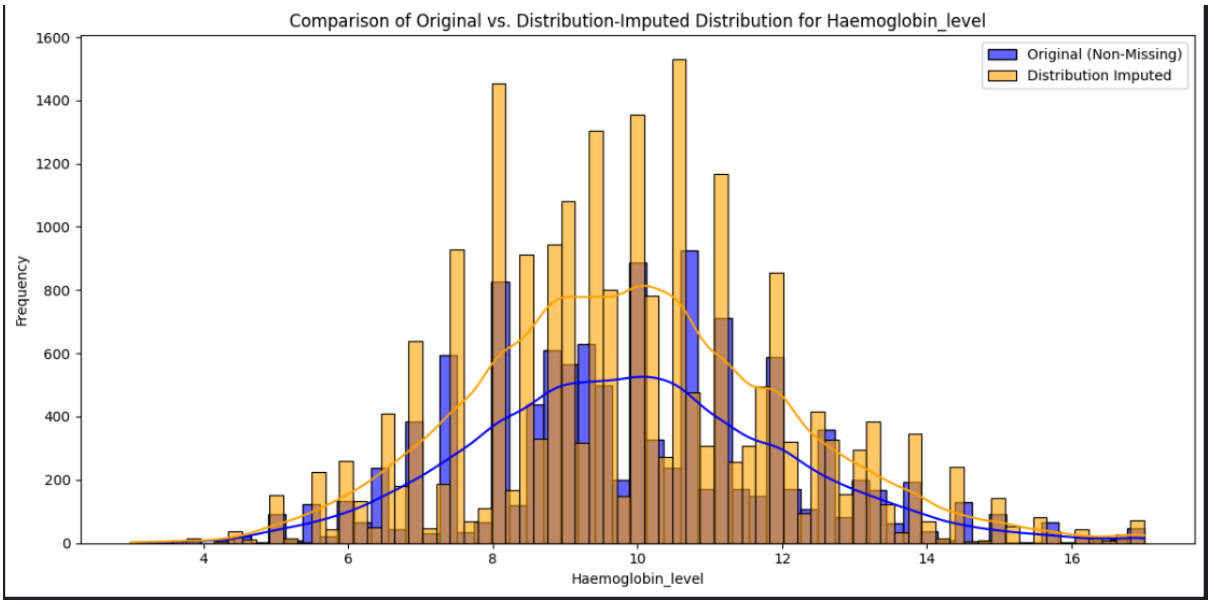
g) Distribution-based imputation

For 'Haemoglobin_level', 'Weight_in_kg', and 'BP_systolic', we will use empirical sampling.Empirical sampling involves drawing random values directly from the observed (non-missing) values of the column's distribution.This approach is robust as it does not assume any specific parametric distribution (e.g., normal, uniform) and preserves the original distribution's shape, central tendency, and spread as much as possible.It's particularly useful when the true underlying distribution is unknown or complex, or when making strong parametric assumptions might lead to biased imputations.

```
print("Missing Data Handling - Distribution-Based Imputation")
selected_dist_impute_cols = ['Haemoglobin_level', 'Weight_in_kg', 'BP_systolic']
df_distribution_imputed=df_cols_deleted.copy()
for col in selected_dist_impute_cols:
    missing_indices=df_distribution_imputed[df_distribution_imputed[col].isnull()].index
    num_missing=len(missing_indices)
    if num_missing>0:
        observed_values=df_distribution_imputed[col].dropna()
sampled_values=observed_values.sample(n=num_missing,replace=True,random_state=42)
        df_distribution_imputed.loc[missing_indices, col]=sampled_values.values
```

```
print("Missing values after distribution-based imputation for selected columns:")
print(df_distribution_imputed[selected_dist_impute_cols].isnull().sum())
print("First 5 rows of df_distribution_imputed (selected columns):")
print(df_distribution_imputed[selected_dist_impute_cols].head())
selected_column_dist_plot='Haemoglobin_level'
plt.figure(figsize=(12,6))
sns.histplot(df_cols_deleted[selected_column_dist_plot].dropna(),kde=True,color='bl
ue',label='Original (Non-Missing)',alpha=0.6)
sns.histplot(df_distribution_imputed[selected_column_dist_plot],kde=True,color='oran
ge',label='Distribution Imputed',alpha=0.6)
plt.title(f'Comparison of Original vs. Distribution-Imputed Distribution for
{selected_column_dist_plot}')
plt.xlabel(selected_column_dist_plot)
plt.ylabel('Frequency')
plt.legend()
plt.tight_layout()
plt.show()
```

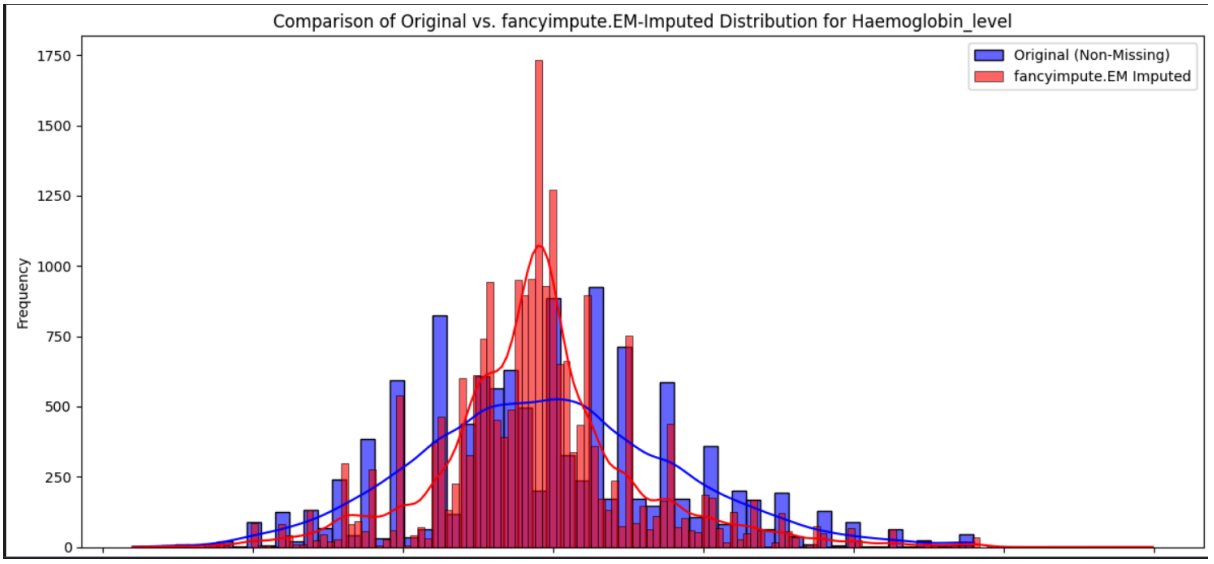
```
Missing Data Handling - Distribution-Based Imputation
Missing values after distribution-based imputation for selected columns:
Haemoglobin_level    0
Weight_in_kg         0
BP_systolic          0
dtype: int64
First 5 rows of df_distribution_imputed (selected columns):
   Haemoglobin_level  Weight_in_kg  BP_systolic
0                8.0           6.2        111.0
1               10.8           6.5        121.0
2                9.1           6.4        131.0
3                8.8           6.3        122.0
4                9.5           6.8        119.0
```



h) EM-based imputation

```
selected_em_fancy_impute_col='Haemoglobin_level'
print(f"Selected target column for fancyimpute IterativeImputer (EM-like) imputation:
{selected_em_fancy_impute_col}")
cols_for_em_fancy_imputation=df_cols_deleted.columns
print(f"Columns participating in fancyimpute.IterativeImputer:
{cols_for_em_fancy_imputation}")
df_em_fancy_imputed=df_cols_deleted.copy()
em_imputer_fancy=IterativeImputer(verbose=False)
imputed_data_fancy=em_imputer_fancy.fit_transform(df_em_fancy_imputed[cols_for
_em_fancy_imputation].values)
df_em_fancy_imputed[cols_for_em_fancy_imputation]=pd.DataFrame(imputed_data
_fancy,columns=cols_for_em_fancy_imputation,index=df_em_fancy_imputed.index)
print(f"Missing values in {selected_em_fancy_impute_col} after
fancyimpute.IterativeImputer:
{df_em_fancy_imputed[selected_em_fancy_impute_col].isnull().sum()}")
print("Missing values in selected columns after fancyimpute.IterativeImputer
imputation:")
print(df_em_fancy_imputed[cols_for_em_fancy_imputation].isnull().sum())
print("First 5 rows of df_em_fancy_imputed (selected columns after
fancyimpute.IterativeImputer imputation):")
print(df_em_fancy_imputed[cols_for_em_fancy_imputation].head())
selected_viz_col = selected_em_fancy_impute_col
plt.figure(figsize=(12,6))
sns.histplot(df_cols_deleted[selected_viz_col].dropna(),kde=True,color='blue',label='
Original (Non-Missing)',alpha=0.6)
sns.histplot(df_em_fancy_imputed[selected_viz_col],kde=True,color='red',label='fanc
yimpute.EM Imputed',alpha=0.6)
plt.title(f'Comparison of Original vs. fancyimpute.EM-Imputed Distribution for
{selected_viz_col}')
plt.xlabel(selected_viz_col)
plt.ylabel('Frequency')
plt.legend()
plt.tight_layout()
plt.show()
```

```
Selected target column for fancyimpute IterativeImputer (EM-like) imputation: Haemoglobin_level
Columns participating in fancyimpute.IterativeImputer: Index(['test_salt_iodine', 'Age', 'date_of_birth', 'month_of_birth',
       'year_of_birth', 'Weight_in_kg', 'Length_height_cm',
       'Haemoglobin_level', 'BP_systolic', 'BP_systolic_2_reading',
       'BP_Diastolic', 'BP_Diastolic_2reading', 'Pulse_rate',
       'Pulse_rate_2_reading', 'fasting_blood_glucose_mg_dl'],
      dtype='object')
Missing values in Haemoglobin_level after fancyimpute.IterativeImputer: 0
Missing values in selected columns after fancyimpute.IterativeImputer imputation:
test_salt_iodine              0
Age                           0
date_of_birth                 0
month_of_birth                0
year_of_birth                 0
Weight_in_kg                  0
Length_height_cm              0
Haemoglobin_level             0
BP_systolic                   0
BP_systolic_2_reading         0
BP_Diastolic                  0
BP_Diastolic_2reading         0
Pulse_rate                    0
Pulse_rate_2_reading          0
fasting_blood_glucose_mg_dl   0
dtype: int64
```

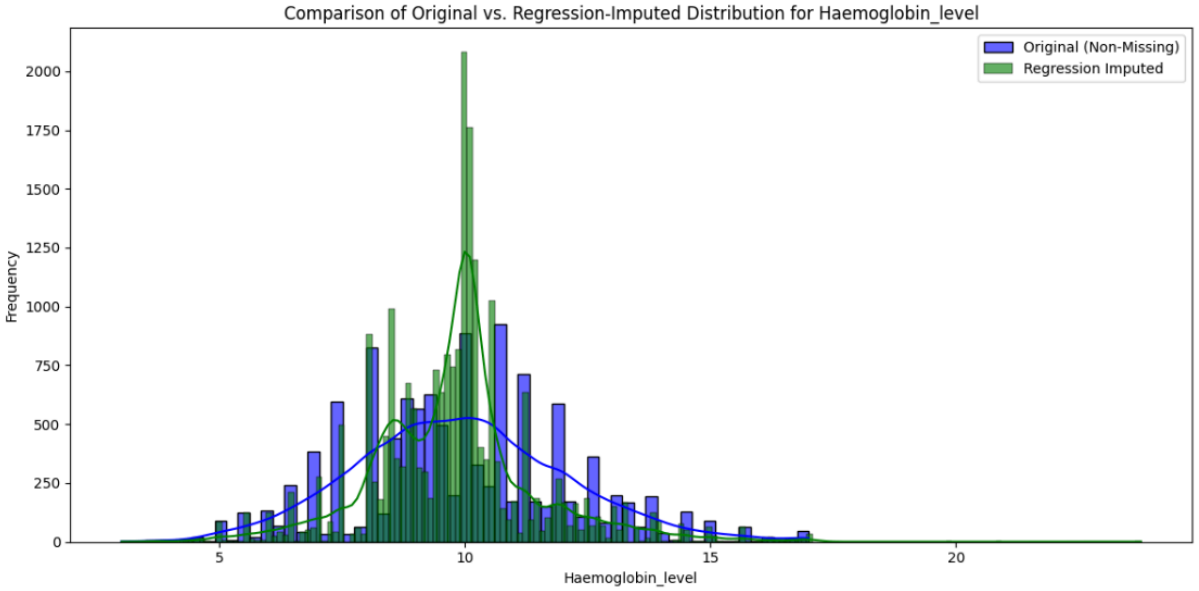


i) Regression-based imputation

```
print("Missing Data Handling - Regression-Based Imputation")
selected_regression_impute_col='Haemoglobin_level'
print(f"Target column for regression imputation: {selected_regression_impute_col}")
df_regression_imputed=df_cols_deleted.copy()
if df_regression_imputed['Length_height_cm'].isnull().any():
    median_len_height=df_regression_imputed['Length_height_cm'].median()

df_regression_imputed['Length_height_cm']=df_regression_imputed['Length_height_
cm'].fillna(median_len_height)
    print(f"'Length_height_cm' missing values filled with median:
{median_len_height:.2f}")
if df_regression_imputed['Weight_in_kg'].isnull().any():
    median_weight = df_regression_imputed['Weight_in_kg'].median()
    df_regression_imputed['Weight_in_kg'] =
df_regression_imputed['Weight_in_kg'].fillna(median_weight)
    print(f"'Weight_in_kg' missing values filled with median: {median_weight:.2f}")
if df_regression_imputed['BP_systolic'].isnull().any():
```

```
    median_bp_systolic = df_regression_imputed['BP_systolic'].median()
    df_regression_imputed['BP_systolic'] =
df_regression_imputed['BP_systolic'].fillna(median_bp_systolic)
    print(f"'BP_systolic' missing values filled with median: {median_bp_systolic:.2f}")
predictor_cols=['Age','Weight_in_kg','Length_height_cm','BP_systolic']
print(f"Predictor columns for regression: {predictor_cols}")
print("Missing values in predictor columns after handling:")
print(df_regression_imputed[predictor_cols].isnull().sum())
known_data=df_regression_imputed[df_regression_imputed[selected_regression_im
pute_col].notna()]
missing_data=df_regression_imputed[df_regression_imputed[selected_regression_i
mpute_col].isna()]
print(f"Shape of known_data (rows with observed {selected_regression_impute_col}):
{known_data.shape}")
print(f"Shape of missing_data (rows with missing {selected_regression_impute_col}):
{missing_data.shape}")
X_train=known_data[predictor_cols]
y_train=known_data[selected_regression_impute_col]
print(f"Shape of X_train (predictors for known data): {X_train.shape}")
print(f"Shape of y_train (target for known data): {y_train.shape}")
X_missing=missing_data[predictor_cols]
print(f"Shape of X_missing (predictors for missing data): {X_missing.shape}")
model=LinearRegression()
model.fit(X_train, y_train)
print("Linear Regression model trained successfully.")
print(f"Model coefficients: {model.coef_}")
print(f"Model intercept: {model.intercept_}")
predicted_values=model.predict(X_missing)
df_regression_imputed.loc[missing_data.index,selected_regression_impute_col]=pre
dicted_values
print(f"Missing values for '{selected_regression_impute_col}' in
df_regression_imputed after prediction:")
print(df_regression_imputed[selected_regression_impute_col].isnull().sum())
plt.figure(figsize=(12,6))
sns.histplot(df_cols_deleted[selected_regression_impute_col].dropna(),kde=True,col
or='blue',label='Original (Non-Missing)',alpha=0.6)
sns.histplot(df_regression_imputed[selected_regression_impute_col],kde=True,color
='green',label='Regression Imputed',alpha=0.6)
plt.title(f'Comparison of Original vs. Regression-Imputed Distribution for
{selected_regression_impute_col}')
plt.xlabel(selected_regression_impute_col)
plt.ylabel('Frequency')
plt.legend()
plt.tight_layout()
plt.show()
```

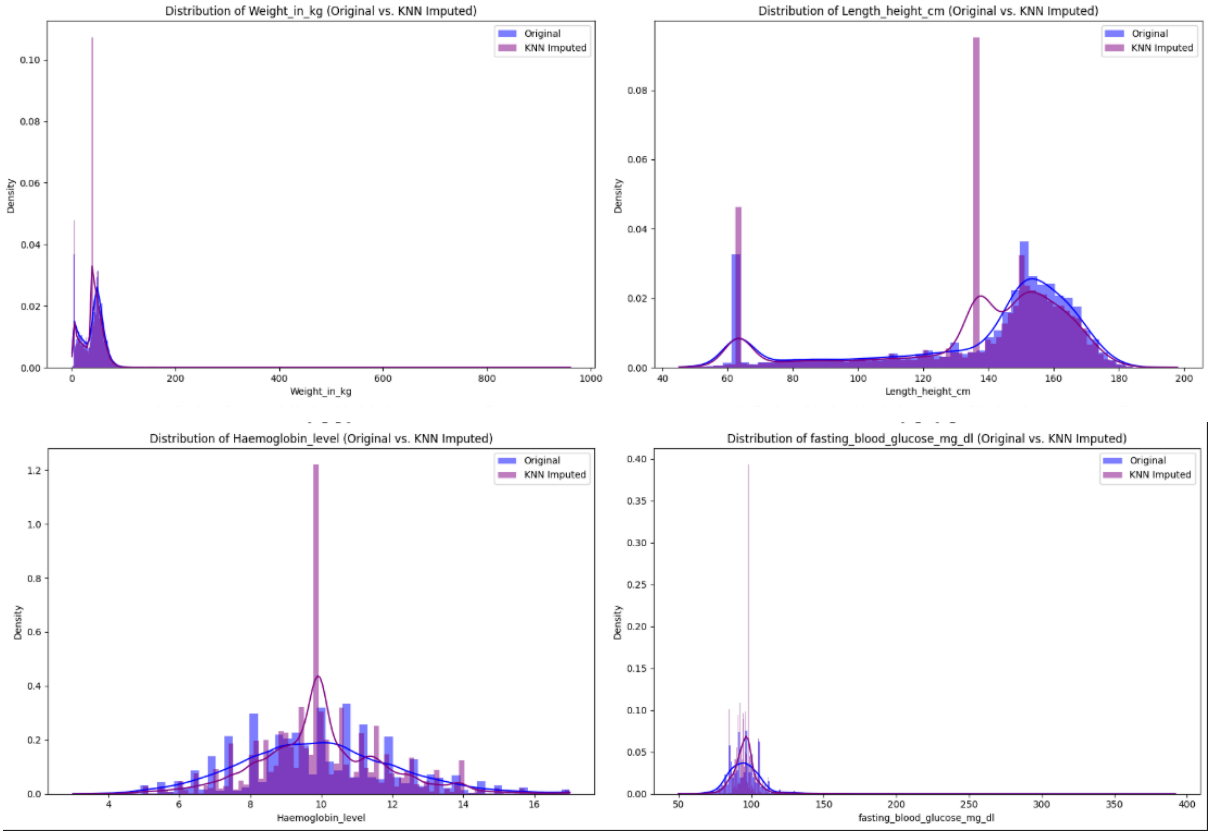
```
Missing Data Handling - Regression-Based Imputation
Target column for regression imputation: Haemoglobin_level
'Length_height_cm' missing values filled with median: 150.00
'Weight_in_kg' missing values filled with median: 45.30
'BP_systolic' missing values filled with median: 121.00
Predictor columns for regression: ['Age', 'Weight_in_kg', 'Length_height_cm', 'BP_systolic']
Missing values in predictor columns after handling:
Age                 0
Weight_in_kg        0
Length_height_cm    0
BP_systolic         0
dtype: int64
Shape of known_data (rows with observed Haemoglobin_level): (11692, 16)
Shape of missing_data (rows with missing Haemoglobin_level): (10407, 16)
Shape of X_train (predictors for known data): (11692, 4)
Shape of y_train (target for known data): (11692,)
Shape of X_missing (predictors for missing data): (10407, 4)
Linear Regression model trained successfully.
Model coefficients: [-0.00931954  0.014852    0.01230062  0.01361257]
Model intercept: 6.070306176384669
Missing values for 'Haemoglobin_level' in df_regression_imputed after prediction:
0
```



j) KNN Imputation

```
print("KNN Imputation")
df_knn_imputed=df.copy()
numerical_df_for_knn=df_knn_imputed[numerical_cols_with_missing].copy()
knn_imputer=KNNImputer(n_neighbors=5)
df_knn_imputed[numerical_cols_with_missing]=knn_imputer.fit_transform(numerical_
df_for_knn)
print("KNN imputation applied to numerical columns.")
selected_numerical_cols=['Weight_in_kg','Length_height_cm','Haemoglobin_level','fa
sting_blood_glucose_mg_dl']
plt.figure(figsize=(18, 12))
for i, col in enumerate(selected_numerical_cols):
    plt.subplot(2, 2, i + 1)
```

```
sns.histplot(df[col].dropna(),color='blue',label='Original',kde=True,stat='density',linewi
dth=0,alpha=0.5)
    sns.histplot(df_knn_imputed[col],color='purple',label='KNN
Imputed',kde=True,stat='density',linewidth=0,alpha=0.5)
    plt.title(f'Distribution of {col} (Original vs. KNN Imputed)')
    plt.legend()
plt.tight_layout()
plt.show()
print("Distribution plots for original and KNN-imputed data displayed.")
```



k) Multiple Imputation

```
print("Missing Data Handling - Advanced Imputation (MICE)")
df_mice_imputed=df_cols_deleted.copy()
cols_for_mice_imputation=numerical_cols_with_missing
print(f"Columns selected for MICE imputation process: {cols_for_mice_imputation}")
mice_imputer=IterativeImputer(BayesianRidge(),max_iter=10,random_state=42)
df_mice_imputed_subset=df_mice_imputed[cols_for_mice_imputation]
imputed_data_mice=mice_imputer.fit_transform(df_mice_imputed_subset)
df_mice_imputed[cols_for_mice_imputation]=pd.DataFrame(imputed_data_mice,
columns=cols_for_mice_imputation, index=df_mice_imputed.index)
print(f"\nMissing values in selected columns after MICE imputation:")
print(df_mice_imputed[cols_for_mice_imputation].isnull().sum())
print("\nFirst 5 rows of df_mice_imputed (selected columns after MICE imputation):")
print(df_mice_imputed[cols_for_mice_imputation].head())
selected_mice_impute_col='Haemoglobin_level'
```

```
plt.figure(figsize=(12,6))
sns.histplot(df_cols_deleted[selected_mice_impute_col].dropna(),kde=True,color='blue',label='Original (Non-Missing)',alpha=0.6)
sns.histplot(df_mice_imputed[selected_mice_impute_col],kde=True,color='green',label='MICE Imputed',alpha=0.6)
plt.title(f'Comparison of Original vs. MICE-Imputed Distribution for {selected_mice_impute_col}')
plt.xlabel(selected_mice_impute_col)
plt.ylabel('Frequency')
plt.legend()
plt.tight_layout()
plt.show()
```

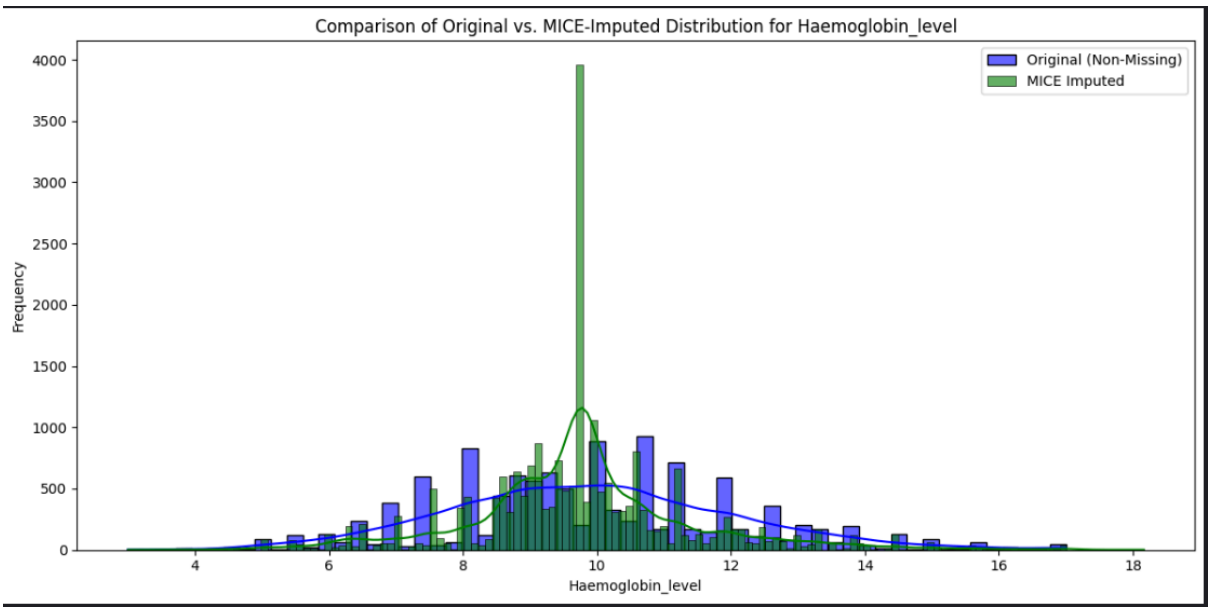
```
Missing Data Handling - Advanced Imputation (MICE)
Columns selected for MICE imputation process: ['Weight_in_kg', 'Length_height_cm', 'Haemoglobin_level', 'BP_systolic', 'BP_systolic_2_reading', 'BP_Diastolic', 'BP_Diastolic_2reading', 'Pulse_rate', 'Pulse_rate_2_reading', 'fasting_blood_glucose_mg_dl']

Missing values in selected columns after MICE imputation:
Weight_in_kg                0
Length_height_cm            0
Haemoglobin_level           0
BP_systolic                 0
BP_systolic_2_reading       0
BP_Diastolic                0
BP_Diastolic_2reading       0
Pulse_rate                  0
Pulse_rate_2_reading        0
fasting_blood_glucose_mg_dl    0
dtype: int64
```

First 5 rows of df_mice_imputed (selected columns after MICE imputation):

	Weight_in_kg	Length_height_cm	Haemoglobin_level	BP_systolic
0	6.2	60.700001	8.620423	130.381375
1	6.5	60.599998	8.622786	130.455538
2	6.4	59.599998	8.611441	130.616537
3	6.3	59.400002	8.608270	130.633752
4	6.8	60.799999	8.628214	130.475783

	BP_systolic_2_reading	BP_Diastolic	BP_Diastolic_2reading	Pulse_rate
0	148.410841	69.481253	70.272516	68.889082
1	148.481413	69.520105	70.310118	68.884462
2	148.832699	69.488225	70.295499	68.775750
3	148.893808	69.471000	70.282494	68.752395
4	148.443168	69.564453	70.348326	68.911851

	Pulse_rate_2_reading	fasting_blood_glucose_mg_dl
0	72.615356	96.092372
1	72.613707	96.144662
2	72.551954	96.181452
3	72.538435	96.176266
4	72.630146	96.181210



Interpretation of Imputation Methods:

a) Best Performing Methods:

MICE, KNN, and EM (Iterative Imputer) performed the best because they use relationships between variables to estimate missing values. This helps preserve the original data distribution, variance, and multivariate relationships, which is important since the dataset mostly has MAR missingness. As a result, the imputed values look realistic and maintain the overall structure of the data.

b) Moderately Effective Methods:

Hot-Deck and Distribution-Based imputation performed reasonably well because they help maintain the original distribution of the data. However, they do not fully capture relationships between variables, which limits their effectiveness compared to model-based methods.

c) Poor Performing Methods:

Mean, Median, and Mode imputation performed poorly because they distort the distribution and reduce variance, creating artificial spikes in the data. List-wise deletion was the worst since it removed all rows due to high missingness, leading to complete data loss. LOCF/NOCB were also unsuitable because they are designed for time-series data, while this dataset is cross-sectional, leading to arbitrary and unreliable imputations.

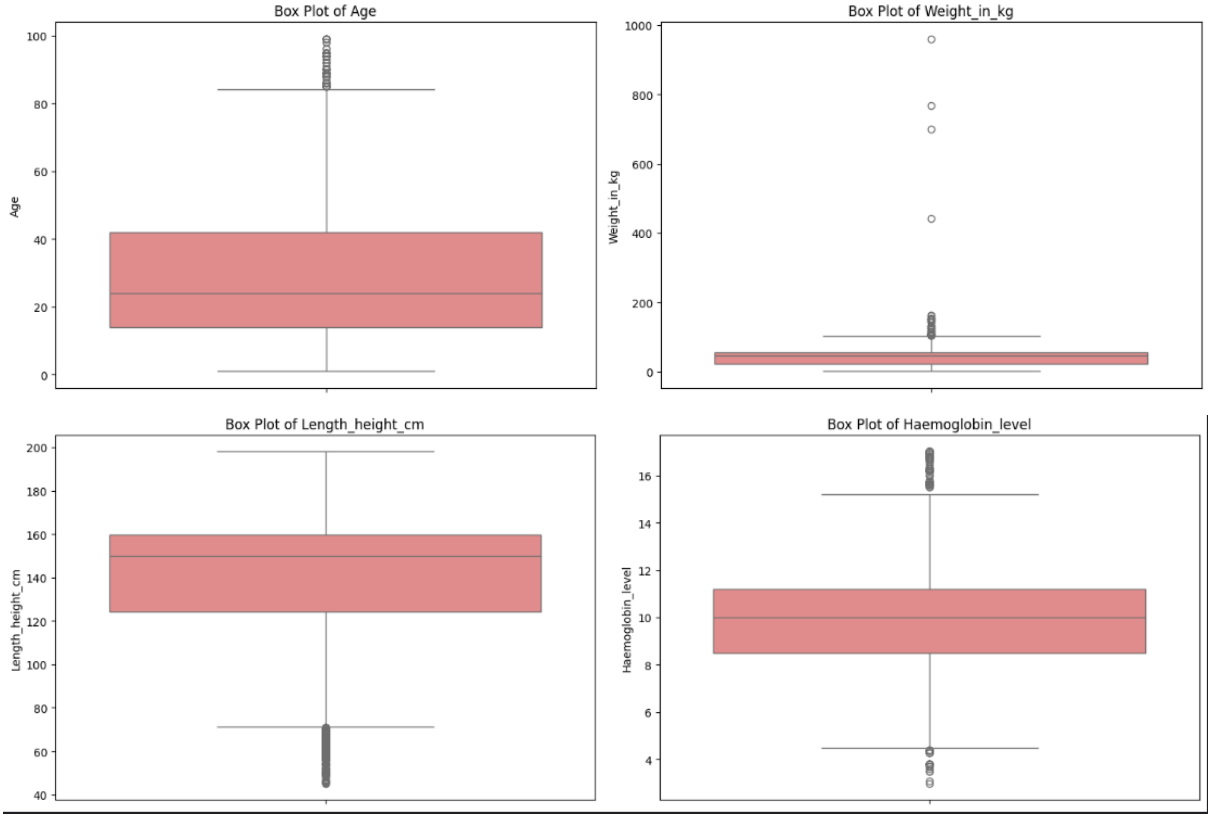
Outlier handling

```
print("Outlier Identification and Visualization")
selected_outlier_cols = ['Age', 'Weight_in_kg', 'Length_height_cm', 'Haemoglobin_level']
print(f"Selected columns for outlier analysis: {selected_outlier_cols}")
print("Outlier Detection using IQR Method")
outliers_count={}
for col in selected_outlier_cols:
```

```
    temp_series = df_cols_deleted[col].dropna()
    Q1=temp_series.quantile(0.25)
    Q3=temp_series.quantile(0.75)
    IQR=Q3-Q1
    lower_bound=Q1-1.5*IQR
    upper_bound=Q3+1.5*IQR
    col_outliers=temp_series[(temp_series<lower_bound)|(temp_series>upper_bound)]
    outliers_count[col]=len(col_outliers)
    print(f"Column: {col}")
    print(f"Q1: {Q1:.2f}")
    print(f"Q3: {Q3:.2f}")
    print(f"IQR: {IQR:.2f}")
    print(f"Lower Bound: {lower_bound:.2f}")
    print(f"Upper Bound: {upper_bound:.2f}")
    print(f"Number of outliers: {outliers_count[col]}")
plt.figure(figsize=(15,10))
for i,col in enumerate(selected_outlier_cols):
    plt.subplot(2,2,i+1)
    sns.boxplot(y=df_cols_deleted[col].dropna(),color='lightcoral')
    plt.title(f'Box Plot of {col}')
    plt.ylabel(col)
    plt.xlabel('')
plt.tight_layout()
plt.show()
```

```
Outlier Identification and Visualization
Selected columns for outlier analysis: ['Age', 'Weight_in_kg', 'Length_height_cm', 'Haemoglobin_level']
Outlier Detection using IQR Method
Column: Age
Q1: 14.00
Q3: 42.00
IQR: 28.00
Lower Bound: -28.00
Upper Bound: 84.00
Number of outliers: 124
Column: Weight_in_kg
Q1: 22.50
Q3: 54.80
IQR: 32.30
Lower Bound: -25.95
Upper Bound: 103.25
Number of outliers: 33
Column: Length_height_cm
Q1: 124.20
Q3: 159.60
IQR: 35.40
Lower Bound: 71.10
Upper Bound: 212.70
Number of outliers: 1936
Column: Haemoglobin_level
Q1: 8.50
Q3: 11.20
IQR: 2.70
Lower Bound: 4.45
```

```
Upper Bound: 15.25
Number of outliers: 181
```



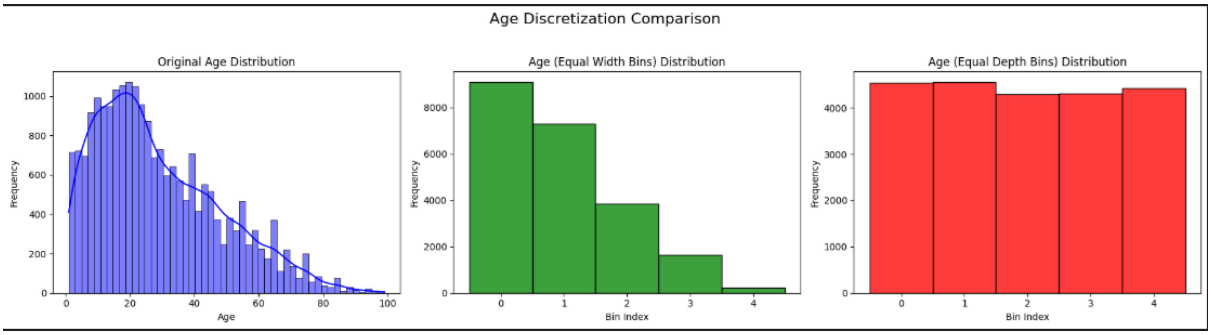
a) Unsupervised discretization

```
print("Outlier Handling - Unsupervised Discretization (Equal Width, Equal Depth)")
df_equal_width_discretized=df_mice_imputed.copy()
num_bins=5
df_equal_width_discretized['Age_Equal_Width_Bin']=pd.cut(df_equal_width_discretized['Age
'],bins=num_bins,labels=False,include_lowest=True)
df_equal_depth_discretized=df_mice_imputed.copy()
df_equal_depth_discretized['Age_Equal_Depth_Bin']=pd.qcut(df_equal_depth_discretized['A
ge'],q=num_bins,labels=False,duplicates='drop')
print(f"Applied Equal Width and Equal Depth Discretization to 'Age' column with {num_bins}
bins/quantiles.")
print("First 5 rows of df_equal_width_discretized (Age and new bin column):")
print(df_equal_width_discretized[['Age','Age_Equal_Width_Bin']].head())
print("First 5 rows of df_equal_depth_discretized (Age and new bin column):")
print(df_equal_depth_discretized[['Age','Age_Equal_Depth_Bin']].head())
num_bins=5
df_equal_width_discretized['Haemoglobin_Equal_Width_Bin']=pd.cut(df_equal_width_discret
ized['Haemoglobin_level'],bins=num_bins,labels=False,include_lowest=True)
df_equal_depth_discretized['Haemoglobin_Equal_Depth_Bin']=pd.qcut(df_equal_depth_disc
retized['Haemoglobin_level'],q=num_bins,labels=False,duplicates='drop')
print(f"Applied Equal Width and Equal Depth Discretization to 'Haemoglobin_level' column
with {num_bins} bins/quantiles.")
print("First 5 rows of df_equal_width_discretized (Haemoglobin_level and new bin column):")
```

```
print(df_equal_width_discretized[['Haemoglobin_level',
'Haemoglobin_Equal_Width_Bin']].head())
print("First 5 rows of df_equal_depth_discretized (Haemoglobin_level and new bin column):")
print(df_equal_depth_discretized[['Haemoglobin_level',
'Haemoglobin_Equal_Depth_Bin']].head())
plt.figure(figsize=(18,5))
plt.subplot(1,3,1)
sns.histplot(df_cols_deleted['Age'],kde=True,color='blue')
plt.title('Original Age Distribution')
plt.xlabel('Age')
plt.ylabel('Frequency')
plt.subplot(1,3,2)
sns.histplot(df_equal_width_discretized['Age_Equal_Width_Bin'].dropna(),bins=num_bins,dis
crete=True,color='green')
plt.title('Age (Equal Width Bins) Distribution')
plt.xlabel('Bin Index')
plt.ylabel('Frequency')
plt.subplot(1,3,3)
sns.histplot(df_equal_depth_discretized['Age_Equal_Depth_Bin'].dropna(),bins=num_bins,di
screte=True,color='red')
plt.title('Age (Equal Depth Bins) Distribution')
plt.xlabel('Bin Index')
plt.ylabel('Frequency')
plt.suptitle('Age Discretization Comparison',fontsize=16)
plt.tight_layout(rect=[0,0.03,1,0.95])
plt.show()
plt.figure(figsize=(18,5))
plt.subplot(1,3,1)
sns.histplot(df_cols_deleted['Haemoglobin_level'].dropna(),kde=True,color='blue')
plt.title('Original Haemoglobin_level Distribution')
plt.xlabel('Haemoglobin_level')
plt.ylabel('Frequency')
plt.subplot(1,3,2)
sns.histplot(df_equal_width_discretized['Haemoglobin_Equal_Width_Bin'].dropna(),bins=nu
m_bins,discrete=True,color='green')
plt.title('Haemoglobin_level (Equal Width Bins) Distribution')
plt.xlabel('Bin Index')
plt.ylabel('Frequency')
plt.subplot(1,3,3)
sns.histplot(df_equal_depth_discretized['Haemoglobin_Equal_Depth_Bin'].dropna(),bins=nu
m_bins,discrete=True,color='red')
plt.title('Haemoglobin_level (Equal Depth Bins) Distribution')
plt.xlabel('Bin Index')
plt.ylabel('Frequency')
plt.suptitle('Haemoglobin_level Discretization Comparison',fontsize=16)
plt.tight_layout(rect=[0,0.03,1,0.95])
plt.show()
```

```
Outlier Handling - Unsupervised Discretization (Equal Width, Equal Depth)
Applied Equal Width and Equal Depth Discretization to 'Age' column with 5 bins/quantiles.
First 5 rows of df_equal_width_discretized (Age and new bin column):
   Age  Age_Equal_Width_Bin
0    7                    0
1    7                    0
2    7                    0
3    7                    0
4    8                    0
First 5 rows of df_equal_depth_discretized (Age and new bin column):
   Age  Age_Equal_Depth_Bin
0    7                    0
1    7                    0
2    7                    0
3    7                    0
4    8                    0
```

```
Applied Equal Width and Equal Depth Discretization to 'Haemoglobin_level' column with 5 bins/quantiles.
First 5 rows of df_equal_width_discretized (Haemoglobin_level and new bin column):
   Haemoglobin_level  Haemoglobin_Equal_Width_Bin
0           8.620423                            1
1           8.622786                            1
2           8.611441                            1
3           8.608270                            1
4           8.628214                            1
First 5 rows of df_equal_depth_discretized (Haemoglobin_level and new bin column):
   Haemoglobin_level  Haemoglobin_Equal_Depth_Bin
0           8.620423                            0
1           8.622786                            0
2           8.611441                            0
3           8.608270                            0
4           8.628214                            0
```

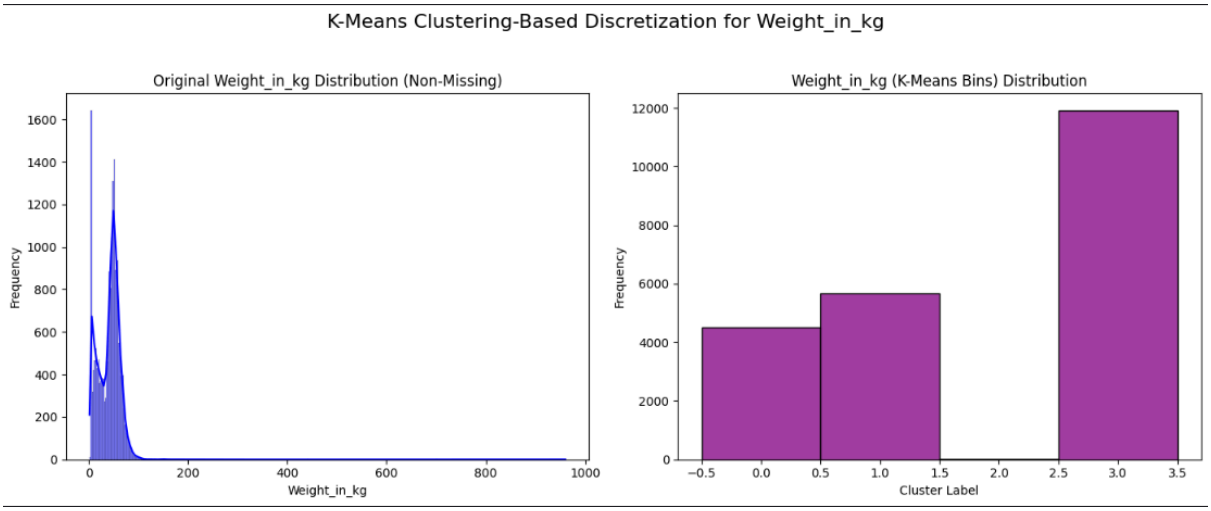


b) K-means clustering

K-Means clustering aims to partition data points into k clusters such that each data point belongs to the cluster with the nearest mean.When applied for discretization, K-Means groups values that are numerically close to each other.For 'Weight_in_kg', which showed some outliers at the higher end in the box plot, if 'k' is chosen appropriately, K-Means can isolate these extreme values into their own cluster (or a small number of clusters).For instance, if there are a few very high 'Weight_in_kg' values that are far from the majority of the data points, K-Means might form a cluster around these extreme values. This effectively 'bins' the outliers separately from the main distribution, allowing them to be treated distinctly in subsequent analysis without heavily influencing the main data bins.This method is 'unsupervised' in that it doesn't use a target variable, but it leverages the inherent structure (density and distance) of the numerical data to create meaningful bins.

```
print("Outlier Handling - Clustering-Based Discretization (K-Means)")
df_kmeans_discretized=df_cols_deleted.copy()
selected_kmeans_col='Weight_in_kg'
print(f"Selected column for K-Means clustering-based discretization:
'{selected_kmeans_col}'")
if df_kmeans_discretized[selected_kmeans_col].isnull().any():
    median_weight=df_kmeans_discretized[selected_kmeans_col].median()
    df_kmeans_discretized[selected_kmeans_col].fillna(median_weight,inplace=True)
    print(f"Missing values in '{selected_kmeans_col}' imputed with median:
{median_weight:.2f}")
n_clusters=4
print(f"Number of clusters (k) chosen for K-Means: {n_clusters}")
data_for_kmeans=df_kmeans_discretized[[selected_kmeans_col]].values
kmeans=KMeans(n_clusters=n_clusters,random_state=42,n_init=10)
df_kmeans_discretized[f'{selected_kmeans_col}_KMeans_Bin']=kmeans.fit_predict(d
ata_for_kmeans)
print("First 5 rows of df_kmeans_discretized (original and K-Means bin column):")
print(df_kmeans_discretized[[selected_kmeans_col,f'{selected_kmeans_col}_KMean
s_Bin']].head())
plt.figure(figsize=(14,6))
plt.subplot(1,2,1)
sns.histplot(df_cols_deleted[selected_kmeans_col].dropna(),kde=True,color='blue')
plt.title(f'Original {selected_kmeans_col} Distribution (Non-Missing)')
plt.xlabel(selected_kmeans_col)
plt.ylabel('Frequency')
plt.subplot(1,2,2)
sns.histplot(df_kmeans_discretized[f'{selected_kmeans_col}_KMeans_Bin'],bins=n_c
lusters,discrete=True,color='purple')
plt.title(f'{selected_kmeans_col} (K-Means Bins) Distribution')
plt.xlabel('Cluster Label')
plt.ylabel('Frequency')
plt.suptitle(f'K-Means Clustering-Based Discretization for
{selected_kmeans_col}',fontsize=16)
plt.tight_layout(rect=[0,0.03,1,0.95])
plt.show()
```

```
Outlier Handling - Clustering-Based Discretization (K-Means)
Selected column for K-Means clustering-based discretization: 'Weight_in_kg'
Missing values in 'Weight_in_kg' imputed with median: 45.30
Number of clusters (k) chosen for K-Means: 4
First 5 rows of df_kmeans_discretized (original and K-Means bin column):
   Weight_in_kg  Weight_in_kg_KMeans_Bin
0           6.2                        1
1           6.5                        1
2           6.4                        1
3           6.3                        1
4           6.8                        1
```



c) **Supervised discretization**

```
df_entropy_discretized=df_mice_imputed.copy()
target=df_entropy_discretized["year_of_birth"].fillna(0)
X=df_entropy_discretized[["Age"]].fillna(df_entropy_discretized["Age"].mean())
tree=DecisionTreeClassifier(max_leaf_nodes=4)
tree.fit(X,target)
thresholds=tree.tree_.threshold
print("Entropy based split thresholds:", thresholds)
```

```
Entropy based split thresholds: [15.5 -2.  21.5 -2.  25.5 -2.  -2. ]
```

Outlier Handling Summary:

Using the IQR method, outliers were identified in all selected variables. Age and Weight_in_kg had relatively fewer outliers, while Length_height_cm and Haemoglobin_level showed a much larger number of extreme values. The box plots confirmed these results, showing many points outside the whiskers, especially for height and haemoglobin levels.

To handle these outliers, discretization techniques were applied. Methods such as Equal Width, Equal Depth, K-Means, and Entropy-based discretization grouped continuous values into bins. This reduces the impact of extreme values because outliers are placed into outer bins or clusters, and all values in a bin are treated equally. As a result, the influence of extreme values is reduced, making the dataset more stable and less sensitive to noise during further analysis or modeling.

Normalization:

a) **MinMax Scaler**

```
df_minmax_scaled=df_mice_imputed.copy()
minmax_scaler=MinMaxScaler()
df_minmax_scaled[df_minmax_scaled.columns]=minmax_scaler.fit_transform(df_min
max_scaled[df_minmax_scaled.columns])
print("Min-Max Scaled DataFrame (df_minmax_scaled) ")
print("Descriptive statistics for 'Weight_in_kg' after Min-Max Scaling:")
```

```
print(df_minmax_scaled['Weight_in_kg'].describe())
print("First 5 rows of df_minmax_scaled (selected numerical columns after scaling):")
print(df_minmax_scaled[df_minmax_scaled.columns].head())
```

```
Min-Max Scaled DataFrame (df_minmax_scaled)
Descriptive statistics for 'Weight_in_kg' after Min-Max Scaling:
count    22099.000000
mean         0.090028
std          0.022547
min          0.000000
25%          0.078419
50%          0.090280
75%          0.102437
max          1.000000
Name: Weight_in_kg, dtype: float64
First 5 rows of df_minmax_scaled (selected numerical columns after scaling):
   test_salt_iodine       Age  date_of_birth  month_of_birth  year_of_birth  \
0          0.233333  0.061224       0.466667        0.166667       1.000000
1          1.000000  0.061224       0.400000        1.000000       0.990385
2          1.000000  0.061224       0.466667        1.000000       0.990385
3          1.000000  0.061224       0.466667        0.750000       0.990385
4          1.000000  0.071429       0.033333        0.083333       1.000000

   Weight_in_kg  Length_height_cm  Haemoglobin_level  BP_systolic  \
0      0.056971          0.356939           0.370670     0.342306
1      0.057267          0.356478           0.370826     0.342694
2      0.057168          0.351860           0.370078     0.343537
3      0.057070          0.350936           0.369869     0.343627
4      0.057564          0.357401           0.371184     0.342800
```

b) Standard Scaler

```
df_standard_scaled=df_mice_imputed.copy()
standard_scaler=StandardScaler()
df_standard_scaled[df_standard_scaled.columns]=standard_scaler.fit_transform(df_s
tandard_scaled[df_standard_scaled.columns])
print("Standard Scaled DataFrame (df_standard_scaled)")
print("Descriptive statistics for 'Weight_in_kg' after Standard Scaling:")
print(df_standard_scaled['Weight_in_kg'].describe())
print("First 5 rows of df_standard_scaled (selected numerical columns after
scaling):")
print(df_standard_scaled[df_standard_scaled.columns].head())
```

```
Standard Scaled DataFrame (df_standard_scaled)
Descriptive statistics for 'Weight_in_kg' after Standard Scaling:
count    2.209900e+04
mean     1.543330e-16
std      1.000023e+00
min     -3.992953e+00
25%     -5.148932e-01
50%      1.115867e-02
75%      5.503617e-01
max      4.035933e+01
Name: Weight_in_kg, dtype: float64
First 5 rows of df_standard_scaled (selected numerical columns after scaling):
   test_salt_iodine       Age  date_of_birth  month_of_birth  year_of_birth  \
0         -2.456102 -1.123387       0.079186       -1.846707       1.491292
1          0.526736 -1.123387      -0.347834        2.556087       1.440330
2          0.526736 -1.123387       0.079186        2.556087       1.440330
3          0.526736 -1.123387       0.079186        1.235249       1.440330
4          0.526736 -1.071932      -2.696445       -2.286987       1.491292

   Weight_in_kg  Length_height_cm  Haemoglobin_level  BP_systolic  \
0     -1.466170         -2.216900          -0.680640     0.477141
1     -1.453019         -2.219868          -0.679230     0.482097
2     -1.457403         -2.249548          -0.685999     0.492856
3     -1.461786         -2.255484          -0.687892     0.494006
4     -1.439868         -2.213932          -0.675991     0.483450
```

c) MaxAbs Scaler

```
df_maxabs_scaled=df_mice_imputed.copy()
maxabs_scaler=MaxAbsScaler()
df_maxabs_scaled[df_maxabs_scaled.columns]=maxabs_scaler.fit_transform(df_ma
xabs_scaled[df_maxabs_scaled.columns])
print("MaxAbs Scaled DataFrame (df_maxabs_scaled)")
print("Descriptive statistics for 'Weight_in_kg' after MaxAbs Scaling:")
print(df_maxabs_scaled['Weight_in_kg'].describe())
print("First 5 rows of df_maxabs_scaled (selected numerical columns after scaling):")
print(df_maxabs_scaled[df_maxabs_scaled.columns].head())
```

```
MaxAbs Scaled DataFrame (df_maxabs_scaled)
Descriptive statistics for 'Weight_in_kg' after MaxAbs Scaling:
count    22099.000000
mean         0.041284
std          0.023755
min         -0.053566
25%          0.029053
50%          0.041550
75%          0.054358
max          1.000000
Name: Weight_in_kg, dtype: float64
First 5 rows of df_maxabs_scaled (selected numerical columns after scaling):
   test_salt_iodine       Age  date_of_birth  month_of_birth  year_of_birth  \
0          0.233333  0.070707       0.483871        0.166667       1.000000
1          1.000000  0.070707       0.419355        1.000000       0.999503
2          1.000000  0.070707       0.483871        1.000000       0.999503
3          1.000000  0.070707       0.483871        0.750000       0.999503
4          1.000000  0.080808       0.064516        0.083333       1.000000

   Weight_in_kg  Length_height_cm  Haemoglobin_level  BP_systolic  \
0      0.006456          0.303555           0.474618     0.509296
1      0.006769          0.303054           0.474748     0.509586
2      0.006665          0.298054           0.474124     0.510215
3      0.006560          0.297053           0.473949     0.510282
4      0.007081          0.304055           0.475047     0.509665
```

d) Robust Scaler

```
df_robust_scaled=df_mice_imputed.copy()
robust_scaler=RobustScaler()
df_robust_scaled[df_robust_scaled.columns]=robust_scaler.fit_transform(df_robust_s
caled[df_robust_scaled.columns])
print("Robust Scaled DataFrame (df_robust_scaled)")
print("Descriptive statistics for 'Weight_in_kg' after Robust Scaling:")
print(df_robust_scaled['Weight_in_kg'].describe())
print("First 5 rows of df_robust_scaled (selected numerical columns after scaling):")
print(df_robust_scaled[df_robust_scaled.columns].head())
```

```
Robust Scaled DataFrame (df_robust_scaled)
Descriptive statistics for 'Weight_in_kg' after Robust Scaling:
count    22099.000000
mean        -0.010475
std          0.938764
min         -3.758829
25%         -0.493827
50%          0.000000
75%          0.506173
max         37.876541
Name: Weight_in_kg, dtype: float64
First 5 rows of df_robust_scaled (selected numerical columns after scaling):
   test_salt_iodine       Age  date_of_birth  month_of_birth  year_of_birth  \
0             -23.0 -0.607143            0.0            -4.0       0.892857
1               0.0 -0.607143           -2.0             6.0       0.857143
2               0.0 -0.607143            0.0             6.0       0.857143
3               0.0 -0.607143            0.0             3.0       0.857143
4               0.0 -0.571429          -13.0            -5.0       0.892857

   Weight_in_kg  Length_height_cm  Haemoglobin_level  BP_systolic  \
0     -1.386831         -3.394904          -0.773120     0.678928
1     -1.374486         -3.398880          -0.771518     0.685980
2     -1.378601         -3.438632          -0.779207     0.701289
3     -1.382716         -3.446583          -0.781357     0.702926
4     -1.362140         -3.390929          -0.767840     0.687905
```

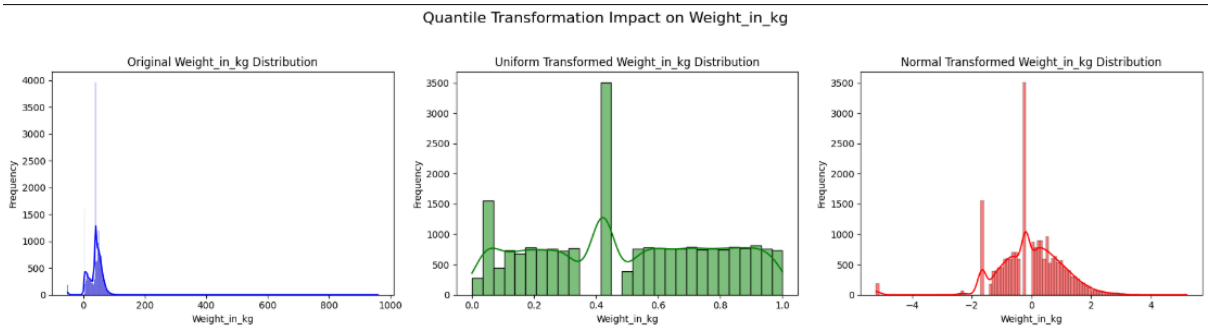
e) Quantile Transformer

```
df_quantile_uniform_scaled=df_mice_imputed.copy()
quantile_uniform_scaler=QuantileTransformer(output_distribution='uniform',random_state=42)
df_quantile_uniform_scaled[df_quantile_uniform_scaled.columns]=quantile_uniform_scaler.fit_transform(df_quantile_uniform_scaled[df_quantile_uniform_scaled.columns])
print("Quantile Transformed (Uniform) DataFrame (df_quantile_uniform_scaled)")
print("Descriptive statistics for 'Weight_in_kg' after Uniform Quantile Transformation:")
print(df_quantile_uniform_scaled['Weight_in_kg'].describe())
print("First 5 rows of df_quantile_uniform_scaled (selected numerical columns after scaling):")
print(df_quantile_uniform_scaled[df_quantile_uniform_scaled.columns].head())
df_quantile_normal_scaled=df_mice_imputed.copy()
quantile_normal_scaler=QuantileTransformer(output_distribution='normal',random_state=42)
df_quantile_normal_scaled[df_quantile_normal_scaled.columns]=quantile_normal_scaler.fit_transform(df_quantile_normal_scaled[df_quantile_normal_scaled.columns])
print("Quantile Transformed (Normal) DataFrame (df_quantile_normal_scaled)")
print("Descriptive statistics for 'Weight_in_kg' after Normal Quantile Transformation:")
print(df_quantile_normal_scaled['Weight_in_kg'].describe())
print("First 5 rows of df_quantile_normal_scaled (selected numerical columns after scaling):")
print(df_quantile_normal_scaled[df_quantile_normal_scaled.columns].head())
selected_viz_col='Weight_in_kg'
plt.figure(figsize=(18, 5))
```

```
plt.subplot(1,3,1)
sns.histplot(df_mice_imputed[selected_viz_col],kde=True,color='blue')
plt.title(f'Original {selected_viz_col} Distribution')
plt.xlabel(selected_viz_col)
plt.ylabel('Frequency')
plt.subplot(1,3,2)
sns.histplot(df_quantile_uniform_scaled[selected_viz_col], kde=True, color='green')
plt.title(f'Uniform Transformed {selected_viz_col} Distribution')
plt.xlabel(selected_viz_col)
plt.ylabel('Frequency')
plt.subplot(1,3,3)
sns.histplot(df_quantile_normal_scaled[selected_viz_col], kde=True, color='red')
plt.title(f'Normal Transformed {selected_viz_col} Distribution')
plt.xlabel(selected_viz_col)
plt.ylabel('Frequency')
plt.suptitle(f'Quantile Transformation Impact on {selected_viz_col}', fontsize=16)
plt.tight_layout(rect=[0,0.03,1,0.95])
plt.show()
```

```
Quantile Transformed (Uniform) DataFrame (df_quantile_uniform_scaled)
Descriptive statistics for 'Weight_in_kg' after Uniform Quantile Transformation:
count    22099.000000
mean         0.502887
std          0.287316
min          0.000000
25%          0.257758
50%          0.503504
75%          0.751752
max          1.000000
Name: Weight_in_kg, dtype: float64
First 5 rows of df_quantile_uniform_scaled (selected numerical columns after scaling):
   test_salt_iodine       Age  date_of_birth  month_of_birth  year_of_birth  \
0          0.041041  0.108108       0.521021        0.060060       1.000000
1          1.000000  0.108108       0.144645        1.000000       0.984985
2          1.000000  0.108108       0.521021        1.000000       0.984985
3          1.000000  0.108108       0.521021        0.893894       0.984985
4          1.000000  0.128128       0.030030        0.022523       1.000000

   Weight_in_kg  Length_height_cm  Haemoglobin_level  BP_systolic  \
0      0.087087          0.015399           0.167076     0.795794
1      0.089089          0.015193           0.167301     0.796748
2      0.088589          0.013714           0.166543     0.797693
3      0.088088          0.013614           0.166354     0.797792
4      0.090249          0.015604           0.168174     0.796891
```

```
   BP_systolic_2_reading  BP_Diastolic  BP_Diastolic_2reading  Pulse_rate  \
0               0.941864      0.209958               0.245956    0.121531
1               0.948070      0.210783               0.248213    0.121450
2               0.949419      0.210121               0.247506    0.120276
3               0.949613      0.209717               0.246641    0.120076
4               0.942558      0.211994               0.249232    0.121933

   Pulse_rate_2_reading  fasting_blood_glucose_mg_dl
0              0.209317                     0.478236
1              0.209255                     0.483852
2              0.208192                     0.487497
3              0.208056                     0.486907
4              0.209869                     0.487470
Quantile Transformed (Normal) DataFrame (df_quantile_normal_scaled)
Descriptive statistics for 'Weight_in_kg' after Normal Quantile Transformation:
count    22099.000000
mean        -0.008776
std          1.070796
min         -5.199338
25%         -0.650274
50%          0.008782
75%          0.680013
max          5.199338
Name: Weight_in_kg, dtype: float64
```



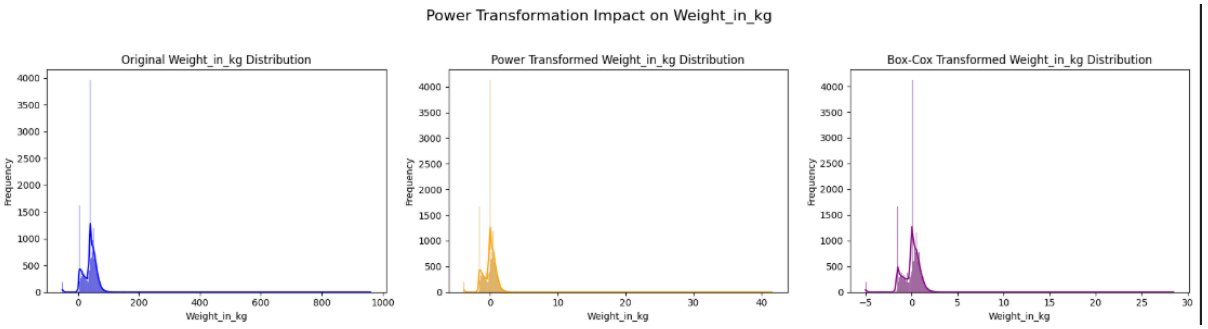
f) Power Transformer

```
df_power_yj_scaled=df_mice_imputed.copy()
power_yj_scaler=PowerTransformer(method='yeo-johnson',standardize=True)
df_power_yj_scaled[df_power_yj_scaled.columns]=power_yj_scaler.fit_transform(df_
power_yj_scaled[df_power_yj_scaled.columns])
print("Power Transformed (Yeo-Johnson) DataFrame (df_power_yj_scaled)")
print("Descriptive statistics for 'Weight_in_kg' after Yeo-Johnson Power
Transformation:")
print(df_power_yj_scaled['Weight_in_kg'].describe())
print("First 5 rows of df_power_yj_scaled (selected numerical columns after
scaling):")
print(df_power_yj_scaled[df_power_yj_scaled.columns].head())
df_power_bc_scaled=df_mice_imputed.copy()
for col in df_power_bc_scaled.columns:
    if (df_power_bc_scaled[col]<=0).any():
        min_val=df_power_bc_scaled[col].min()
        if min_val<=0:
            epsilon=1e-6
            df_power_bc_scaled[col]=df_power_bc_scaled[col]+abs(min_val)+epsilon
            print(f"Adjusted column '{col}' for Box-Cox transformation due to non-positive
values (min value: {min_val:.2f})")
```

```
power_bc_scaler=PowerTransformer(method='box-cox',standardize=True)
df_power_bc_scaled[df_power_bc_scaled.columns]=power_bc_scaler.fit_transform(d
f_power_bc_scaled[df_power_bc_scaled.columns])
print("Power Transformed (Box-Cox) DataFrame (df_power_bc_scaled)")
print("Descriptive statistics for 'Weight_in_kg' after Box-Cox Power Transformation:")
print(df_power_bc_scaled['Weight_in_kg'].describe())
print("First 5 rows of df_power_bc_scaled (selected numerical columns after
scaling):")
print(df_power_bc_scaled[df_power_bc_scaled.columns].head())
selected_viz_col='Weight_in_kg'
plt.figure(figsize=(18,5))
plt.subplot(1,3,1)
sns.histplot(df_mice_imputed[selected_viz_col],kde=True,color='blue')
plt.title(f'Original {selected_viz_col} Distribution')
plt.xlabel(selected_viz_col)
plt.ylabel('Frequency')
plt.subplot(1,3,2)
sns.histplot(df_power_yj_scaled[selected_viz_col],kde=True,color='orange')
plt.title(f'Power Transformed {selected_viz_col} Distribution')
plt.xlabel(selected_viz_col)
plt.ylabel('Frequency')
plt.subplot(1,3,3)
sns.histplot(df_power_bc_scaled[selected_viz_col],kde=True,color='purple')
plt.title(f'Box-Cox Transformed {selected_viz_col} Distribution')
plt.xlabel(selected_viz_col)
plt.ylabel('Frequency')
plt.suptitle(f'Power Transformation Impact on {selected_viz_col}', fontsize=16)
plt.tight_layout(rect=[0,0.03,1,0.95])
plt.show()
```

```
Power Transformed (Yeo-Johnson) DataFrame (df_power_yj_scaled)
Descriptive statistics for 'Weight_in_kg' after Yeo-Johnson Power Transformation:
count    2.209900e+04
mean     3.086660e-17
std      1.000023e+00
min     -3.820427e+00
25%     -5.198390e-01
50%      6.846165e-03
75%      5.485938e-01
max      4.158025e+01
Name: Weight_in_kg, dtype: float64
First 5 rows of df_power_yj_scaled (selected numerical columns after scaling):
   test_salt_iodine       Age  date_of_birth  month_of_birth  year_of_birth  \
0         -1.935420 -1.291795       0.049763       -1.886781       1.892378
1          0.542692 -1.291795      -0.382780        2.487229       1.803427
2          0.542692 -1.291795       0.049763        2.487229       1.803427
3          0.542692 -1.291795       0.049763        1.225881       1.803427
4          0.542692 -1.187600      -2.501109       -2.368943       1.892378

   Weight_in_kg  Length_height_cm  Haemoglobin_level  BP_systolic  \
0     -1.464285         -2.058150          -0.663395     0.532791
1     -1.451354         -2.059494          -0.661910     0.537555
2     -1.455665         -2.072775          -0.669041     0.547885
3     -1.459975         -2.075397          -0.671035     0.548989
4     -1.438417         -2.056804          -0.658500     0.538854
```

```
   BP_systolic_2_reading  BP_Diastolic  BP_Diastolic_2reading  Pulse_rate  \
0               1.272598     -0.524225              -0.395399   -0.967764
1               1.275903     -0.520087              -0.391260   -0.968378
2               1.292329     -0.523483              -0.392869   -0.982829
3               1.295183     -0.525318              -0.394301   -0.985938
4               1.274112     -0.515367              -0.387058   -0.964742

   Pulse_rate_2_reading  fasting_blood_glucose_mg_dl
0             -0.596291                     0.038933
1             -0.596527                     0.044499
2             -0.605383                     0.048410
3             -0.607323                     0.047859
4             -0.594172                     0.048384
Adjusted column 'test_salt_iodine' for Box-Cox transformation due to non-positive values (min value: 0.00)
Adjusted column 'month_of_birth' for Box-Cox transformation due to non-positive values (min value: 0.00)
Adjusted column 'Weight_in_kg' for Box-Cox transformation due to non-positive values (min value: -51.44)
Adjusted column 'Length_height_cm' for Box-Cox transformation due to non-positive values (min value: -16.60)
Power Transformed (Box-Cox) DataFrame (df_power_bc_scaled)
Descriptive statistics for 'Weight_in_kg' after Box-Cox Power Transformation:
count    2.209900e+04
mean    -1.851996e-16
std      1.000023e+00
min     -5.015629e+00
25%     -4.891982e-01
50%      4.073378e-02
75%      5.686416e-01
max      2.846565e+01
Name: Weight_in_kg, dtype: float64
```



g) Unit Vector Normalization

```
df_unit_vector_scaled=df_mice_imputed.copy()
normalizer=Normalizer()
df_unit_vector_scaled[df_unit_vector_scaled.columns]=normalizer.fit_transform(df_u
nit_vector_scaled[df_unit_vector_scaled.columns])
print("Unit Vector Scaled DataFrame (df_unit_vector_scaled)")
print("Descriptive statistics for 'Weight_in_kg' after Unit Vector Scaling:")
print(df_unit_vector_scaled['Weight_in_kg'].describe())
print("\nFirst 5 rows of df_unit_vector_scaled (selected numerical columns after
scaling):")
print(df_unit_vector_scaled[df_unit_vector_scaled.columns].head())
```

```
Unit Vector Scaled DataFrame (df_unit_vector_scaled)
Descriptive statistics for 'Weight_in_kg' after Unit Vector Scaling:
count    22099.000000
mean         0.019778
std          0.011265
min         -0.026345
25%          0.013798
50%          0.020164
75%          0.026084
max          0.428716
Name: Weight_in_kg, dtype: float64

First 5 rows of df_unit_vector_scaled (selected numerical columns after scaling):
   test_salt_iodine       Age  date_of_birth  month_of_birth  year_of_birth  \
0          0.003445  0.003445       0.007382        0.000984       0.991221
1          0.014770  0.003446       0.006401        0.005908       0.991094
2          0.014770  0.003446       0.007385        0.005908       0.991087
3          0.014770  0.003446       0.007385        0.004431       0.991096
4          0.014764  0.003937       0.000984        0.000492       0.991132
```

Summary of Scaling and Transformation:

The scaling and transformation techniques helped standardize the feature values and reduce the effect of large magnitude differences between variables, which improves the performance of many machine learning models. Methods that normalize the scale of data (such as range-based scaling) were useful for ensuring that no single feature dominates the model due to its larger numeric range.

Techniques that center data around a common distribution (such as mean-based or distribution-based transformations) helped make the dataset more suitable for algorithms that assume normally distributed inputs. These methods improved the comparability between features and stability of model training.

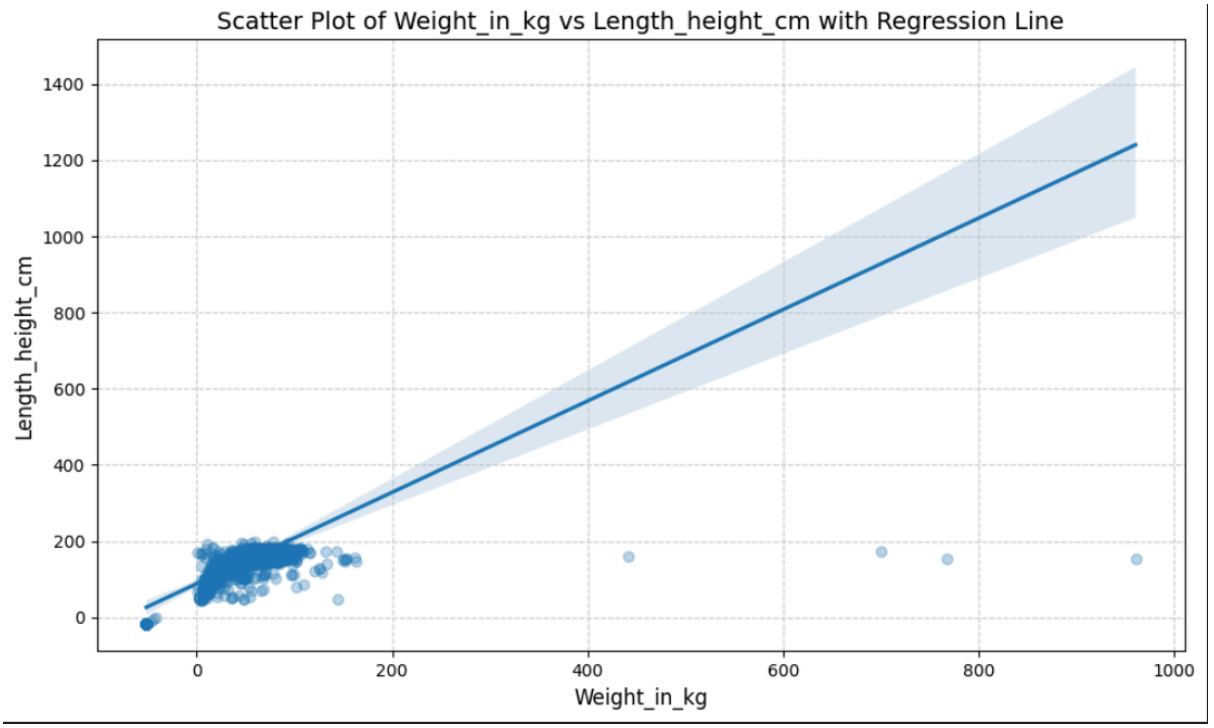
Methods that use median and IQR based scaling were particularly useful because they reduced the influence of outliers, making them more effective for datasets where extreme values are present. Similarly, power and quantile transformations helped reduce skewness in the data and reshape the distribution, which can improve model performance for algorithms sensitive to data distribution.

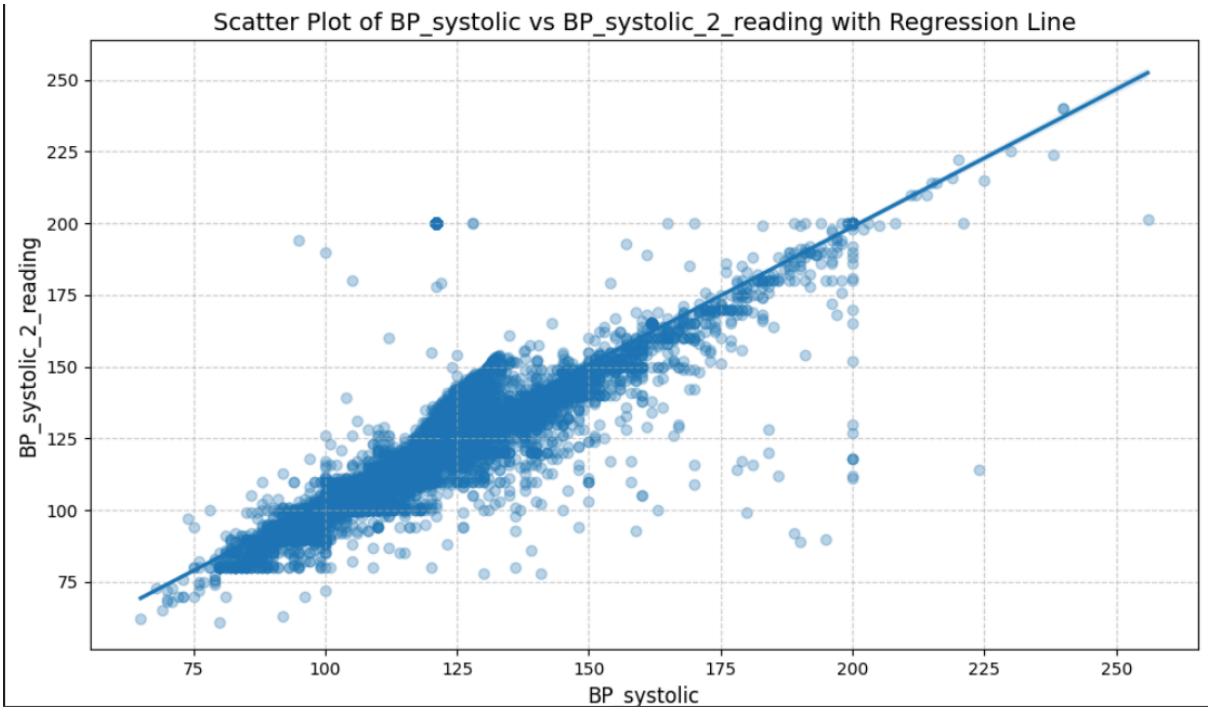
Overall, outlier-robust scaling and distribution-based transformations were the most effective, as they handled both extreme values and skewed distributions, while simpler range-based scaling methods were useful mainly for basic normalization when the data does not contain strong outliers.

Correlation & Covariance:

a) Scatter plots

```
print("Scatter Plot Analysis for Variable Relationships")
selected_pairs=[('Weight_in_kg','Length_height_cm'),('BP_systolic','BP_systolic_2_reading'),('Age','fasting_blood_glucose_mg_dl'),('Age','Haemoglobin_level')]
for col1, col2 in selected_pairs:
    plt.figure(figsize=(10,6))
sns.regplot(x=df_mice_imputed[col1],y=df_mice_imputed[col2],scatter_kws={'alpha': 0.3})
    plt.title(f'Scatter Plot of {col1} vs {col2} with Regression Line', fontsize=14)
    plt.xlabel(col1,fontsize=12)
    plt.ylabel(col2,fontsize=12)
    plt.grid(True,linestyle='--',alpha=0.6)
    plt.tight_layout()
    plt.show()
```

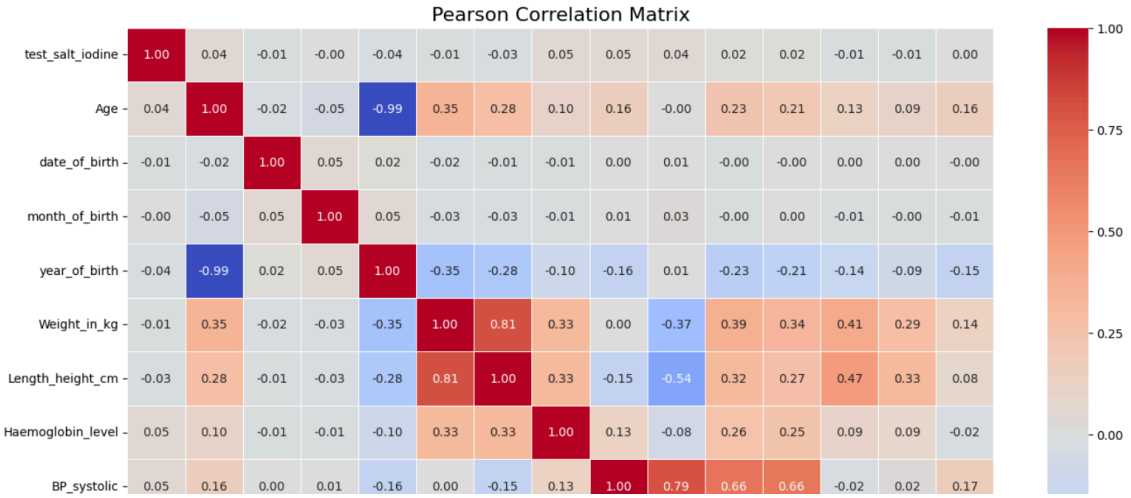




b) Pearson's coefficient

```
print("Pearson Correlation Analysis")
pearson_correlation_matrix=df_mice_imputed.corr(method='pearson')
print("Pearson Correlation Matrix for Numerical Columns in df_fully_imputed:\n")
print(pearson_correlation_matrix)
plt.figure(figsize=(16,12))
sns.heatmap(pearson_correlation_matrix,annot=True,cmap='coolwarm',fmt=".2f",line
widths=.5)
plt.title('Pearson Correlation Matrix',fontsize=16)
plt.show()
```

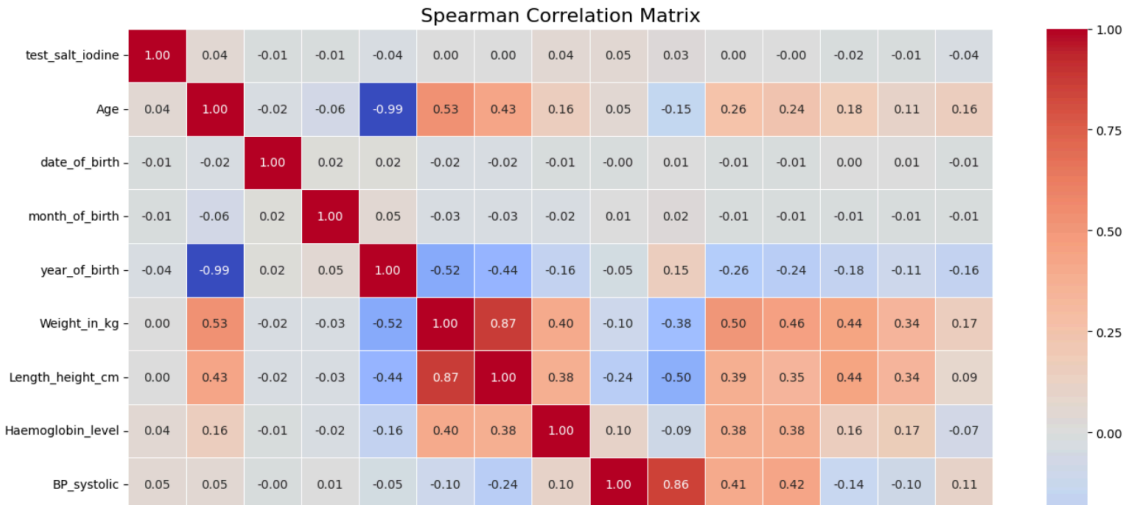
```
Pearson Correlation Matrix for Numerical Columns in df_fully_imputed:

                           test_salt_iodine       Age  date_of_birth  \
test_salt_iodine                   1.000000  0.038578      -0.007491
Age                                0.038578  1.000000      -0.020354
date_of_birth                     -0.007491 -0.020354       1.000000
month_of_birth                    -0.001146 -0.052930       0.050366
year_of_birth                     -0.040194 -0.990571       0.018015
Weight_in_kg                      -0.006282  0.353164      -0.019606
Length_height_cm                  -0.034434  0.278928      -0.011091
Haemoglobin_level                  0.049423  0.100954      -0.006702
BP_systolic                        0.049086  0.164239       0.000465
BP_systolic_2_reading              0.039607 -0.001374       0.005253
BP_Diastolic                       0.019892  0.230295      -0.004349
BP_Diastolic_2reading              0.015396  0.212422      -0.001917
Pulse_rate                        -0.012905  0.133379       0.002070
Pulse_rate_2_reading              -0.006334  0.092222       0.004965
fasting_blood_glucose_mg_dl        0.002812  0.156862      -0.002004
```



c) Spearson's coefficient

```
print("Spearman Correlation Analysis")
spearman_correlation_matrix=df_mice_imputed.corr(method='spearman')
print("Spearman Correlation Matrix for Numerical Columns in df_fully_imputed:\n")
print(spearman_correlation_matrix)
plt.figure(figsize=(16, 12))
sns.heatmap(spearman_correlation_matrix,annot=True,cmap='coolwarm',fmt=".2f",lin
ewidths=.5)
plt.title('Spearman Correlation Matrix',fontsize=16)
plt.show()
```

```
Spearman Correlation Matrix for Numerical Columns in df_fully_imputed:

                           test_salt_iodine       Age  date_of_birth  \
test_salt_iodine                   1.000000  0.037402      -0.008940
Age                                0.037402  1.000000      -0.020993
date_of_birth                     -0.008940 -0.020993       1.000000
month_of_birth                    -0.006651 -0.060540       0.018218
year_of_birth                     -0.038583 -0.992571       0.018318
Weight_in_kg                       0.004464  0.525204      -0.023569
Length_height_cm                   0.000055  0.434998      -0.020012
Haemoglobin_level                  0.043315  0.160185      -0.005517
BP_systolic                        0.048489  0.049341      -0.004902
BP_systolic_2_reading              0.032451 -0.146158       0.006427
BP_Diastolic                       0.004225  0.260247      -0.008107
BP_Diastolic_2reading             -0.001826  0.236725      -0.008405
Pulse_rate                        -0.016488  0.180695       0.004869
Pulse_rate_2_reading              -0.011735  0.109745       0.006720
fasting_blood_glucose_mg_dl       -0.040616  0.164991      -0.006741
```



d) Regression-based correlation

```
print("Regression-Based Correlation Analysis")
selected_regression_pairs=[('Weight_in_kg','Length_height_cm'),('BP_systolic','BP_s
ystolic_2_reading'),('Age','Haemoglobin_level')]
for col1,col2 in selected_regression_pairs:
    print(f"Linear Regression for {col2} ~ {col1}")
    X=df_mice_imputed[[col1]]
    y=df_mice_imputed[col2]
    model=LinearRegression()
    model.fit(X,y)
    r_squared=model.score(X,y)
    print(f"R-squared value: {r_squared:.4f}")
```

```
Regression-Based Correlation Analysis
Linear Regression for Length_height_cm ~ Weight_in_kg
R-squared value: 0.6610
Linear Regression for BP_systolic_2_reading ~ BP_systolic
R-squared value: 0.6288
Linear Regression for Haemoglobin_level ~ Age
R-squared value: 0.0102
```

e) Partial & Multiple correlation

```
X_partial=df_mice_imputed[['Age','Length_height_cm']]
y_partial=df_mice_imputed['Weight_in_kg']
X_partial=sm.add_constant(X_partial)
partial_model=sm.OLS(y_partial, X_partial).fit()
print("Partial Correlation (Age vs Weight controlling Height)")
print(partial_model.summary())
X_multi=df_mice_imputed[['Age','Length_height_cm','Pulse_rate']]
y_multi=df_mice_imputed['Weight_in_kg']
X_multi=sm.add_constant(X_multi)
multi_model=sm.OLS(y_multi, X_multi).fit()
print("\nMultiple Correlation Model")
print(multi_model.summary())
```

```
Partial Correlation (Age vs Weight controlling Height)
                            OLS Regression Results
==============================================================================
Dep. Variable:          Weight_in_kg   R-squared:                       0.678
Model:                           OLS   Adj. R-squared:                  0.678
Method:                Least Squares   F-statistic:                 2.330e+04
Date:               Thu, 05 Mar 2026   Prob (F-statistic):               0.00
Time:                       08:57:30   Log-Likelihood:                -87934.
No. Observations:              22099   AIC:                         1.759e+05
Df Residuals:                  22096   BIC:                         1.759e+05
Df Model:                          2
Covariance Type:           nonrobust
==============================================================================
                 coef    std err          t      P>|t|      [0.025      0.975]
------------------------------------------------------------------------------
const        -36.0152      0.362    -99.509      0.000     -36.725     -35.306
Age            0.1609      0.005     34.494      0.000       0.152       0.170
Length_height_cm     0.5246      0.003    194.999      0.000       0.519       0.530
==============================================================================
Omnibus:                    58244.441   Durbin-Watson:                   1.783
Prob(Omnibus):                  0.000   Jarque-Bera (JB):       3153113161.162
Skew:                          30.834   Prob(JB):                         0.00
Kurtosis:                    1852.472   Cond. No.                         593.
==============================================================================

Notes:
[1] Standard Errors assume that the covariance matrix of the errors is correctly specified.
```

```
Multiple Correlation Model
                            OLS Regression Results
==============================================================================
Dep. Variable:          Weight_in_kg   R-squared:                       0.679
Model:                           OLS   Adj. R-squared:                  0.679
Method:                Least Squares   F-statistic:                 1.559e+04
Date:               Thu, 05 Mar 2026   Prob (F-statistic):               0.00
Time:                       08:57:30   Log-Likelihood:                -87905.
No. Observations:              22099   AIC:                         1.758e+05
Df Residuals:                  22095   BIC:                         1.759e+05
Df Model:                          3
Covariance Type:           nonrobust
==============================================================================
                 coef    std err          t      P>|t|      [0.025      0.975]
------------------------------------------------------------------------------
const        -40.9726      0.744    -55.101      0.000     -42.430     -39.515
Age            0.1608      0.005     34.527      0.000       0.152       0.170
Length_height_cm     0.5140      0.003    170.057      0.000       0.508       0.520
Pulse_rate     0.0824      0.011      7.629      0.000       0.061       0.104
==============================================================================
Omnibus:                    58271.287   Durbin-Watson:                   1.784
Prob(Omnibus):                  0.000   Jarque-Bera (JB):       3166541416.588
Skew:                          30.871   Prob(JB):                         0.00
Kurtosis:                    1856.408   Cond. No.                     1.39e+03
==============================================================================

Notes:
[1] Standard Errors assume that the covariance matrix of the errors is correctly specified.
[2] The condition number is large, 1.39e+03. This might indicate that there are
```

f) Covariance Analysis

```
print("Covariance Analysis")
covariance_matrix=df_mice_imputed.cov()
print("Covariance Matrix for Numerical Columns in df_fully_imputed:\n")
print(covariance_matrix)
```

```
Covariance Matrix for Numerical Columns in df_fully_imputed:

                            test_salt_iodine         Age  date_of_birth  \
test_salt_iodine                   59.458788    5.781492      -0.270540
Age                                 5.781492  377.723830      -1.852770
date_of_birth                      -0.270540   -1.852770      21.937293
month_of_birth                     -0.020075   -2.336506       0.535808
year_of_birth                      -6.081769 -377.772674       1.655666
Weight_in_kg                       -1.105078  156.576430      -2.094835
Length_height_cm                   -8.946186  182.651340      -1.750257
Haemoglobin_level                   0.638705    3.288334      -0.052607
BP_systolic                         5.664069   47.766706       0.032600
BP_systolic_2_reading               5.525557   -0.482970       0.445101
BP_Diastolic                        1.624028   47.389254      -0.215655
BP_Diastolic_2reading               1.182204   41.111362      -0.089426
Pulse_rate                         -0.909098   23.682600       0.088575
Pulse_rate_2_reading               -0.389855   14.306058       0.185625
fasting_blood_glucose_mg_dl         0.346947   48.776126      -0.150194

                            month_of_birth  year_of_birth  Weight_in_kg  \
test_salt_iodine                 -0.020075      -6.081769     -1.105078
Age                              -2.336506    -377.772674    156.576430
date_of_birth                     0.535808       1.655666     -2.094835
month_of_birth                    5.158968       2.080030     -1.476039
year_of_birth                     2.080030     385.048681   -158.849892
Weight_in_kg                     -1.476039    -158.849892    520.385511
Length_height_cm                 -2.561005    -188.022368    624.889825
Haemoglobin_level                -0.042322      -3.392074     12.451269
BP_systolic                       0.238456     -47.403624      1.072827
```

Summary of Correlation Analysis :

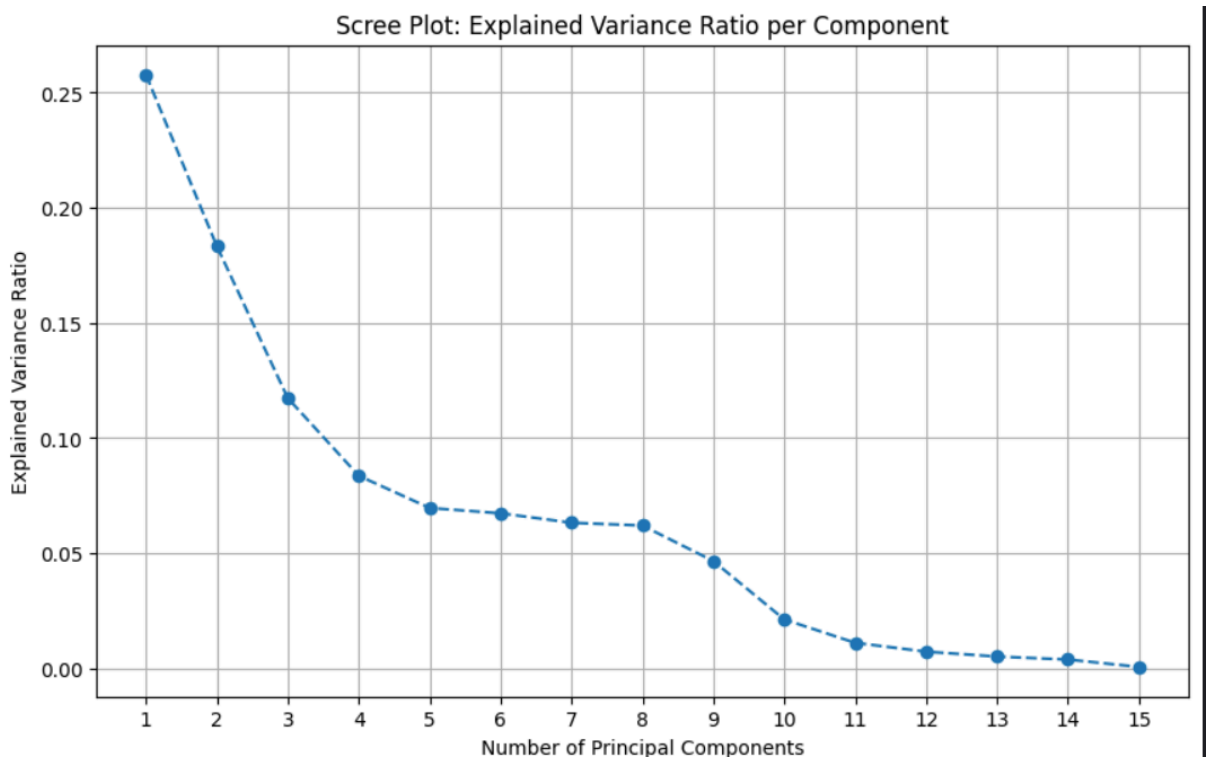
1. Strong Positive Relationships:
    a) Weight_in_kg and Length_height_cm show a strong positive correlation, meaning taller individuals generally weigh more.
    b) Repeated measurements such as BP_systolic & BP_systolic_2_reading, BP_Diastolic & BP_Diastolic_2reading, and Pulse_rate & Pulse_rate_2_reading have very high correlations, indicating consistent medical measurements.
2. Strong Negative Relationship:
    a) Age and year_of_birth have an almost perfect negative correlation because as age increases, year of birth decreases.
3. Moderate Relationships:
    a) Age with Weight_in_kg and Length_height_cm shows moderate correlation, suggesting body growth trends with age.
4. Weak or No Relationships:
    a) fasting_blood_glucose_mg_dl and Haemoglobin_level have very weak correlations with most other variables, indicating they are largely independent features in this dataset.
5. Overall Insight:
    a) Height–weight and repeated health measurements show the strongest correlations, while glucose and haemoglobin levels show minimal relationships with other features.

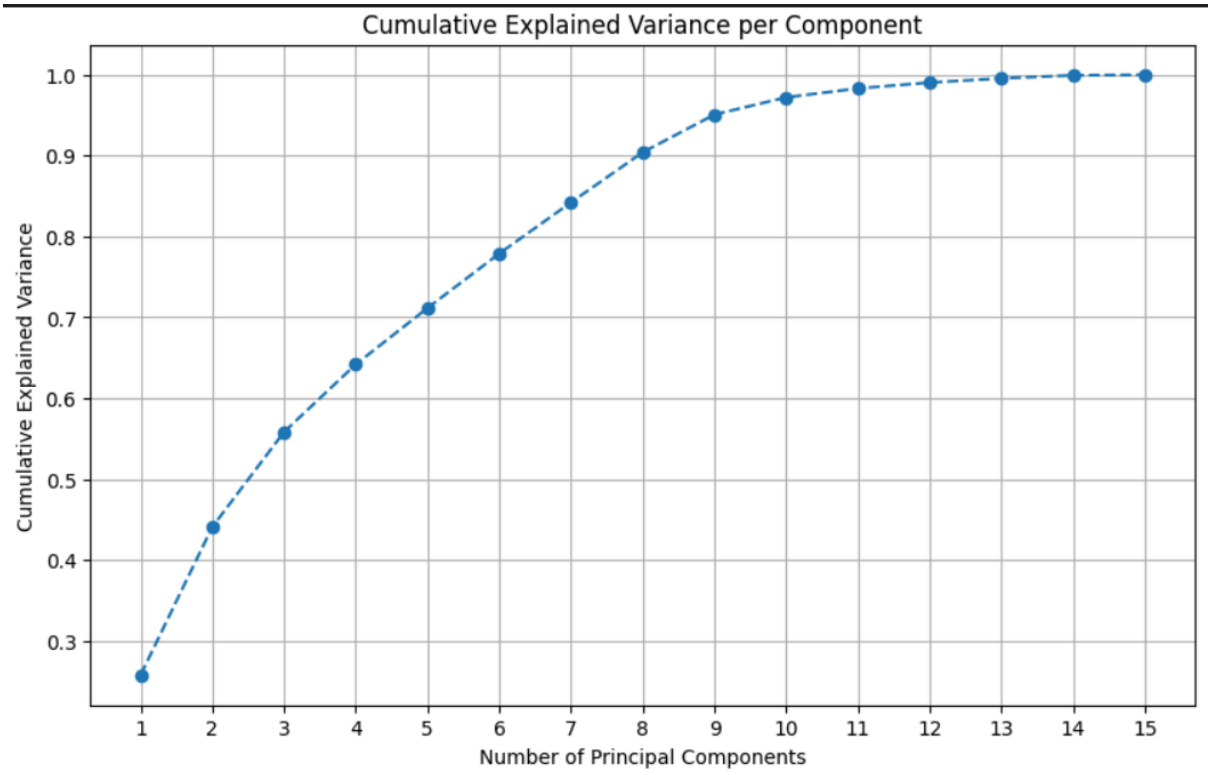
Dimensionality reduction:

a) Principal Component Analysis

```
print("Principal Component Analysis (PCA)\n")
numerical_cols_for_scaling=df_mice_imputed.columns
X_scaled = df_standard_scaled[numerical_cols_for_scaling]
pca=PCA()
pca.fit(X_scaled)
explained_variance_ratio=pca.explained_variance_ratio_
cumulative_explained_variance=np.cumsum(explained_variance_ratio)
print("Explained Variance Ratio per Principal Component:")
for i,ratio in enumerate(explained_variance_ratio):
    print(f"Principal Component {i+1}: {ratio:.4f}")
print("Cumulative Explained Variance:")
for i, cumulative_ratio in enumerate(cumulative_explained_variance):
    print(f"  Up to PC {i+1}: {cumulative_ratio:.4f}")
plt.figure(figsize=(10,6))
plt.plot(range(1,len(explained_variance_ratio)+1),explained_variance_ratio,marker='o',linesty
le='--')
plt.xlabel('Number of Principal Components')
plt.ylabel('Explained Variance Ratio')
plt.title('Scree Plot: Explained Variance Ratio per Component')
plt.grid(True)
plt.xticks(range(1,len(explained_variance_ratio)+1))
plt.show()
plt.figure(figsize=(10, 6))
plt.plot(range(1,
len(cumulative_explained_variance)+1),cumulative_explained_variance,marker='o',linestyle=
'--')
plt.xlabel('Number of Principal Components')
plt.ylabel('Cumulative Explained Variance')
plt.title('Cumulative Explained Variance per Component')
plt.grid(True)
plt.xticks(range(1, len(cumulative_explained_variance)+1))
plt.show()
n_components_optimal=3
print(f"Applying PCA with {n_components_optimal} components for further analysis\n")
pca_optimal=PCA(n_components=n_components_optimal)
pca_optimal.fit(X_scaled)
principal_components=pca_optimal.transform(X_scaled)
pc_column_names=[f'PC{i+1}' for i in range(n_components_optimal)]
df_pca_components=pd.DataFrame(data=principal_components,
columns=pc_column_names)
print("First 5 rows of df_pca_components:")
print(df_pca_components.head())
loadings=pca_optimal.components_.T
df_loadings=pd.DataFrame(loadings,columns=pc_column_names,index=numerical_cols_for
_scaling)
```

```
Principal Component Analysis (PCA)

Explained Variance Ratio per Principal Component:
Principal Component 1: 0.2578
Principal Component 2: 0.1833
Principal Component 3: 0.1173
Principal Component 4: 0.0836
Principal Component 5: 0.0696
Principal Component 6: 0.0673
Principal Component 7: 0.0632
Principal Component 8: 0.0620
Principal Component 9: 0.0466
Principal Component 10: 0.0212
Principal Component 11: 0.0111
Principal Component 12: 0.0073
Principal Component 13: 0.0051
Principal Component 14: 0.0038
Principal Component 15: 0.0006
Cumulative Explained Variance:
  Up to PC 1: 0.2578
  Up to PC 2: 0.4411
  Up to PC 3: 0.5584
  Up to PC 4: 0.6420
  Up to PC 5: 0.7117
  Up to PC 6: 0.7790
  Up to PC 7: 0.8422
  Up to PC 8: 0.9042
  Up to PC 9: 0.9508
  Up to PC 10: 0.9721
```


Scree Plot: Explained Variance Ratio per Component
Explained Variance Ratio
0.25
0.20
0.15
0.10
0.05
0.00
1
2
3
4
5
6
7
8
9
10
11
12
13
14
15
Number of Principal Components


Cumulative Explained Variance per Component
Cumulative Explained Variance
1.0
0.9
0.8
0.7
0.6
0.5
0.4
0.3
1
2
3
4
5
6
7
8
9
10
11
12
13
14
15
Number of Principal Components

```
Applying PCA with 3 components for further analysis

First 5 rows of df_pca_components:
        PC1       PC2       PC3
0 -2.906174  2.102098 -0.821089
1 -2.929515  2.329508 -0.914500
2 -2.950656  2.363191 -0.922448
3 -2.929602  2.336541 -0.827299
4 -2.796756  2.191034 -0.487220
Loadings of Original Features on Principal Components (first 3):
                                PC1       PC2       PC3
test_salt_iodine           0.013512  0.039880  0.057930
Age                        0.289064 -0.028280  0.572118
date_of_birth             -0.008698  0.006445 -0.037262
month_of_birth            -0.020561  0.024529 -0.070617
year_of_birth             -0.289734  0.030887 -0.570864
Weight_in_kg               0.366556 -0.255351  0.039980
Length_height_cm           0.334866 -0.349494 -0.020604
Haemoglobin_level          0.196214 -0.045164 -0.027567
BP_systolic                0.217807  0.498332 -0.027488
BP_systolic_2_reading      0.022774  0.562398 -0.026181
BP_Diastolic               0.399202  0.269127 -0.151458
BP_Diastolic_2reading      0.381918  0.295202 -0.153591
Pulse_rate                 0.312611 -0.222773 -0.358961
Pulse_rate_2_reading       0.279530 -0.163358 -0.388428
fasting_blood_glucose_mg_dl  0.133308  0.078163  0.081126
```

Principal Component 1 (PC1): PC1 has significant positive loadings from 'Weight_in_kg', 'Length_height_cm', and 'Age'. It also shows positive loadings for 'BP_systolic', 'BP_Diastolic', and 'Pulse_rate' related columns. This component appears to capture aspects related to Physical Stature and General Health Metrics, particularly body measurements and some vital signs that tend to increase with age.
Principal Component 2 (PC2): PC2 shows very strong positive loadings from all 'BP_systolic', 'BP_systolic_2_reading', 'BP_Diastolic', 'BP_Diastolic_2reading', 'Pulse_rate', and 'Pulse_rate_2_reading' related columns. This component strongly represents Cardiovascular Health Metrics, specifically the measured blood pressure and pulse rates. It suggests that these readings vary together independently of the first component's influence.
Principal Component 3 (PC3): PC3 has strong positive loadings from 'date_of_birth' and negative loadings from 'year_of_birth' and 'Age'. This component clearly captures Age and Birth Year Information. It differentiates individuals based on their age-related attributes.

b) Independent Component Analysis

```
print("Independent Component Analysis (ICA)")
n_components_ica=3
print(f"Applying ICA with {n_components_ica} components.")
ica=FastICA(n_components=n_components_ica,random_state=42,max_iter=1000)
independent_components=ica.fit_transform(X_scaled)
ic_column_names=[f'IC{i+1}' for i in range(n_components_ica)]
df_ica_components=pd.DataFrame(data=independent_components,columns=ic_column_names)
```

```
print("First 5 rows of df_ica_components:")
print(df_ica_components.head())
mixing_matrix=ica.mixing_
df_mixing_matrix=pd.DataFrame(mixing_matrix, columns=ic_column_names,
index=X_scaled.columns)
print("Mixing Matrix (Original Features' Contributions to Independent Components):")
print(df_mixing_matrix)
```

```
Independent Component Analysis (ICA)
Applying ICA with 3 components.
First 5 rows of df_ica_components:
        IC1       IC2       IC3
0 -1.602500  1.264483 -0.087643
1 -1.704078  1.314291 -0.193047
2 -1.720027  1.329309 -0.202226
3 -1.651508  1.353480 -0.186808
4 -1.383449  1.404868 -0.127823
Mixing Matrix (Original Features' Contributions to Independent Components):
                                IC1       IC2       IC3
test_salt_iodine           0.057941  0.067188 -0.055800
Age                        0.920396  0.068117 -0.222756
date_of_birth             -0.052021 -0.011306 -0.004064
month_of_birth            -0.108284 -0.004582 -0.017868
year_of_birth             -0.920896 -0.063842  0.220598
Weight_in_kg               0.524016 -0.638474 -0.139181
Length_height_cm           0.470105 -0.741015  0.006725
Haemoglobin_level          0.185314 -0.286995 -0.197746
BP_systolic               -0.042869  0.228863 -0.901823
BP_systolic_2_reading     -0.264194  0.513901 -0.734089
BP_Diastolic               0.107682 -0.295568 -0.869911
BP_Diastolic_2reading      0.076257 -0.251696 -0.880869
Pulse_rate                 0.022633 -0.840009 -0.187010
Pulse_rate_2_reading      -0.069430 -0.765352 -0.224637
fasting_blood_glucose_mg_dl  0.184268 -0.012990 -0.250939
```

Dimensionality Reduction Insights:

1. Variance Captured:
    a) PC1 explains 25.78% of variance and the first 3 PCs explain ~55.84%, meaning most information is captured by a few components.
2. Component Patterns:
    a) PC1 is mainly influenced by weight, height, and age (body size related features).
    b) PC2 is dominated by blood pressure measurements.
    c) PC3 captures smaller variations such as glucose or haemoglobin related differences

Feature Engineering:

```
df_base=df_mice_imputed.copy()
initial_cols=set(df_base.columns)
if ("Weight_in_kg" in df_base.columns) and ("Length_height_cm" in df_base.columns):
```

```
    df_base["BMI"]=df_base["Weight_in_kg"]/((df_base["Length_height_cm"]/100)**2)
else:
    df_base["BMI"]=np.nan
    print("Warning: BMI not computed (missing Weight_in_kg or Length_height_cm).")
if "Age" in df_base.columns:
    df_base["Age_months"]=df_base["Age"]*12
else:
    df_base["Age_months"]=np.nan
    print("Warning: Age_months not computed (missing Age).")
if "Age" in df_base.columns:
    max_age=int(np.nanmax(df_base["Age"].dropna())) if df_base["Age"].notna().any() else
100
    bins=[0,13,19,36,61,max_age+1]
    age_labels=['Child (0-12)','Adolescent (13-18)','Young Adult (19-35)','Adult (36-60)','Senior
(61+)']
    if len(age_labels)!=len(bins)-1:
        age_labels=[f'Age Group {i+1}' for i in range(len(bins)-1)]
    df_base["Age_Group"]=pd.cut(df_base["Age"], bins=bins, labels=age_labels, right=False)
else:
    df_base["Age_Group"]=np.nan
nutrition_candidates=["water_month", "ani_milk_month", "semisolid_month_or_day",
"solid_month", "vegetables_month_or_day"]
nutrition_cols=[c for c in nutrition_candidates if c in df_base.columns]
if nutrition_cols:
    df_base["nutrition_score"]=df_base[nutrition_cols].mean(axis=1,skipna=True)
else:
    df_base["nutrition_score"]=np.nan
hb="Haemoglobin_level" if "Haemoglobin_level" in df_base.columns else None
fg="fasting_blood_glucose_mg_dl" if "fasting_blood_glucose_mg_dl" in df_base.columns
else None
if hb or fg:
    hb_series=df_base[hb].fillna(df_base[hb].mean()) if hb else 0
    fg_series=df_base[fg].fillna(df_base[fg].mean()) if fg else 0
    df_base["health_risk"]=hb_series + fg_series
else:
    df_base["health_risk"]=np.nan
    print("Warning: neither Haemoglobin_level nor fasting_blood_glucose_mg_dl present;
health_risk = NaN.")
if ("Age" in df_base.columns) and ("BMI" in df_base.columns):
    df_base["Age_BMI_Interaction"]=df_base["Age"]*df_base["BMI"]
else:
    df_base["Age_BMI_Interaction"]=np.nan
cols=["BMI", "Age_months", "Age_Group", "nutrition_score", "health_risk",
"Age_BMI_Interaction"]
df=df_base[[c for c in cols if c in df_base.columns]].copy()
df
```

Group 1 created: 6 engineered features (BMI, Age_BMI_Interaction, Age_months, Age_Group, nutrition_score, health_risk).
BMI: Body Mass Index calculated from weight and height to estimate body fat level.
Age_BMI_Interaction: Product of age and BMI capturing how body composition effects vary with age.
Age_months: Age converted from years to months for finer age granularity.
Age_Group: Categorical grouping of individuals into age ranges (child, adolescent, adult, etc.).
nutrition_score: A composite indicator estimating nutritional status using variables like BMI, haemoglobin, or weight.
health_risk: Derived indicator estimating potential health risk based on factors such as age, BMI, BP, or glucose.

```
df_base=df_mice_imputed.copy()
initial_cols=set(df_base.columns)
systolic_1="BP_systolic" if "BP_systolic" in df_base.columns else None
systolic_2="BP_systolic_2_reading" if "BP_systolic_2_reading" in df_base.columns else None
diastolic_1="BP_Diastolic" if "BP_Diastolic" in df_base.columns else None
diastolic_2="BP_Diastolic_2reading" if "BP_Diastolic_2reading" in df_base.columns else None
pulse_1="Pulse_rate" if "Pulse_rate" in df_base.columns else None
pulse_2="Pulse_rate_2_reading" if "Pulse_rate_2_reading" in df_base.columns else None
if systolic_1:
    df_base["BP_systolic_avg"]=(df_base[systolic_1] + df_base[systolic_2]) / 2 if systolic_2 else df_base[systolic_1]
else:
    df_base["BP_systolic_avg"]=np.nan
    print("Warning: BP_systolic not present -> BP_systolic_avg = NaN.")
if diastolic_1:
    df_base["BP_diastolic_avg"]=(df_base[diastolic_1] + df_base[diastolic_2]) / 2 if diastolic_2 else df_base[diastolic_1]
else:
    df_base["BP_diastolic_avg"]=np.nan
    print("Warning: BP_Diastolic not present -> BP_diastolic_avg = NaN.")
if systolic_1 and diastolic_1:
    df_base["BP_Difference"]=df_base[systolic_1] - df_base[diastolic_1]
else:
    df_base["BP_Difference"]=np.nan
if pulse_1:
    df_base["Pulse_avg"]=(df_base[pulse_1] + df_base[pulse_2]) / 2 if pulse_2 else df_base[pulse_1]
else:
    df_base["Pulse_avg"]=np.nan
    print("Note: Pulse_rate missing -> Pulse_avg = NaN.")
if pulse_1 and pulse_2:
    df_base["Pulse_Rate_Ratio"]=np.where(df_base[pulse_1] != 0, df_base[pulse_2] / df_base[pulse_1], np.nan)
```

```
else:
    df_base["Pulse_Rate_Ratio"]=np.nan
cols=["BP_systolic_avg", "BP_diastolic_avg", "BP_Difference", "Pulse_avg",
"Pulse_Rate_Ratio"]
df=df_base[[c for c in cols if c in df_base.columns]].copy()
df
```

Group 2 created: 5 engineered features (BP_systolic_avg, BP_diastolic_avg, BP_Difference, Pulse_avg, Pulse_Rate_Ratio).
BP_systolic_avg: Average of multiple systolic blood pressure readings for a more reliable systolic BP value.
BP_diastolic_avg: Average of multiple diastolic blood pressure readings to reduce measurement variation.
BP_Difference: Difference between systolic and diastolic blood pressure indicating pulse pressure.
Pulse_avg: Average of multiple pulse rate measurements for a stable heart rate estimate.
Pulse_Rate_Ratio: Ratio of the second pulse reading to the first to analyze variation between measurements.